



GROUP ASSIGNMENT

CT107-3-3-TXSA

TEXT ANALYTICS AND SENTIMENT ANALYSIS

CSSE__CT107-3-3-TXSA-L-3__2025-12-12

HAND OUT DATE: 10th January 2025

HAND IN DATE: 27th March 2025

WEIGHTAGE: 50%

INSTRUCTIONS TO CANDIDATES:

Submit your assignment via Moodle

Students are advised to underpin their answers with the use of references (cited using the 7th Edition of APA Referencing Style).

Late submissions will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.

Cases of plagiarism will be penalized.

You must obtain 50% overall to pass this module.

NAME	TP NO
MUHAMMAD IRFAN BIN MOHD RIZAL	TP078491
SARVEIN RAO A/L SATHIAH	TP071496
VOON KAI WEN	TP077157
RAMANEISS PILLAI A/L S.GOPALAN	TP070818

Table of Contents

1.0 Introduction.....	10
1.1 Background of Text Analytics	10
1.2 Assignment Overview.....	10
2.0 Part A: Text Pre-processing and Basic NLP Techniques	11
2.1 (Q1) Tokenization and Stop Words Filtering	11
2.1.1 Word Tokenization using Split Function, Regular Expressions and NLTK	11
2.1.2 Comparison of Tokenization Methods.....	12
2.1.3 Stop Words and Punctuation Removal	12
2.1.4 Importance of Removing Stop Words and Punctuation	14
2.2 (Q2) Word Stemming	14
2.2.1 Importance of Stemming in Text Analytics.....	14
2.2.2 Implementation and Output of Regular Expression, Porter Stemmer and Lancaster Stemmer	15
2.2.3 Differences among Regular Expression stemmer, Porter Stemmer and Lancaster Stemmer	18
2.3 (Q3) Parts of Speech Tagging and Syntactic Analysis	22
2.3.1 POS Tagging Demonstration	22
2.3.2 Comparison of POS Tagging Methods	25
2.3.3 Parse Tree Generation.....	27
2.4 (Q4) Sentence Probability using Bigram Models	30
2.4.1 Unsmoothed Bigram Model.....	30
2.4.2 Smoothed Bigram Model.....	31
2.4.3 Python Implementation of Bigram Models.....	32
2.4.4 Results and Analysis	35
2.5 (Q5) Alternative Approach Implementation.....	36
2.5.1 Alternative Tokenization Method: Scikit-Learn (Muhammad Irfan bin Mohd Rizal – TP078491).....	36

2.5.2 Alternative Tokenization Method: spaCy (Ramaneiss – TP070818)	38
2.5.3 Alternative Tokenization Method: HuggingFace (Voon Kai Wen – TP077157)...	41
2.5.4 Alternative Tokenization Method: TextBlob (Sarvein – TP071496)	44
3.0 Part B: Implementation of Text Analytics Techniques and Methodologies.....	46
3.1 (Q1) Dataset and Exploratory Data Analysis.....	46
3.1.1 Dataset Description.....	46
3.1.2 Problem Definition.....	46
3.1.3 Literature Review.....	47
3.1.4 Justification of Dataset Selection.....	50
3.2 Exploratory Data Analysis (EDA)	51
3.2.1 Dataset Overview	51
3.2.2 Vocabulary Analysis.....	52
3.2.3 Overall Summary	53
3.2.6 Chosen Predictive Models	53
3.3 Pre-processing.....	54
3.3.1 Remove Duplicate Data	54
3.3.2 Final Dataset Summary.....	54
3.4 (Q2) Supervised Text Classification.....	55
3.4.1 Support Vector Machine using LinearSVC	55
3.4.2 Evaluation Measures.....	58
3.4.3 Model Evaluation and Results	60
3.4.4 (Q3) Hyper parameters	61
3.5 Long Short-Term Memory (LSTM) Model.....	65
3.5.1 Data Preparation.....	65
3.5.2 Baseline LSTM Model Development	68
3.5.3 Baseline Model Evaluation	70
3.5.4 (Q3) Hyperparameter Tuning Setup	72

3.5.5 Hyperparameter Tuning Using RandomizedSearchCV	73
3.5.6 Tuned Model Evaluation.....	75
3.5.7 Comparison of Baseline and Tuned Models	77
3.6 Logistic Regression Model	78
3.6.1 Data Preparation.....	78
3.6.2 TF-IDF Vectorization Preparation	81
3.6.3 Baseline Logistic Regression Model Development.....	82
3.6.4 Baseline Logistic Regression Model Results Evaluation	84
3.6.5 (Q3) Hyperparameter Tuning using RandomizedSearchCV	88
3.6.6 Tuned Model Evaluation.....	93
3.6.7 Baseline Model and Tuned Model Comparison	95
3.7 Random Forest Model.....	97
3.7.1 Train/Test Split	98
3.7.2 TF-IDF Vectorisation.....	99
3.7.3 Random Forest Model Configuration	100
3.7.4 Evaluation Measures	101
3.7.5 Baseline Model Evaluation Results	102
3.7.6 (Q3) Hyperparameter Selection & Tuning	104
3.7.7 Interpretation of Best Parameters.....	107
3.7.8 Tuned Model Evaluation Results.....	108
3.8 (Q4) Model Evaluation Results	110
3.8.1 Performance Comparison.....	110
3.8.2 Result Analysis	112
4.0 Workload Matrix.....	114
5.0 References.....	115

List of Figures

Figure 1: Importing Libraries.....	11
Figure 2: Tokenization Approaches Output.....	12
Figure 3: Stop Words and Punctuation Removal Code	13
Figure 4: Stop Words and Punctuation Removal Output.....	13
Figure 5: Read the Dataset.....	15
Figure 6: Dataset Output Confirmation	15
Figure 7: Initialization of Stemmers & Word Tokenization.....	16
Figure 8: Word Tokenization Output.....	16
Figure 9: Sorting Unique Words.....	16
Figure 10: Sorted Unique Words Output	16
Figure 11: Stemmer Implementation and Result Table Code.....	17
Figure 12: Stemmer Implementation and Result Table Output	17
Figure 13: POS Tagging using NLTK code	22
Figure 14: POS Tagging Output (NLTK).....	22
Figure 15: POS Tagging using TextBlob Code	23
Figure 16: POS Tagging Output (TextBlob)	23
Figure 17: POS Tagging using Regular Expression Tagger Code	24
Figure 18: POS Tagging Output (Regular Expression Tagger).....	24
Figure 19: Chunk Parser Code.....	27
Figure 20: Parse Tree Output (Chunk Parser).....	27
Figure 21: Context-Free Grammar Parser Code	28
Figure 22: Parse Tree Output (Context-Free Grammar Parser).....	29
Figure 23: Data Import.....	32
Figure 24: Text Corpus Pre-Processing	32
Figure 25: Data Transformation.....	33
Figure 26: Sentence Probability.....	34
Figure 27: Results	34
Figure 28: Initialization and CountVectorizer Tokenization Code.....	36
Figure 29: CountVectorizer Tokenization Output	37
Figure 30: Importing spaCy libraries and Read Dataset.....	38
Figure 31: Dataset Confirmation Output	38
Figure 32: spaCy Tokenization Code	39

Figure 33: spaCy Tokenization Output.....	39
Figure 34: HuggingFace Transformer Installation	41
Figure 35: Installation Process and Succession	41
Figure 36: HuggingFace Tokenization Code.....	42
Figure 37: HuggingFace Tokenization Output Success.....	42
Figure 38: TextBlob Initialization and Word Tokenization	44
Figure 39: Word Tokenization Output.....	45
Figure 40: Dataset Overview	51
Figure 41: Visualization Charts	52
Figure 42: EDA Summary Table	53
Figure 43: Duplicate Data Removal Code	54
Figure 44: Final Dataset Summary	54
Figure 45: Initializing Libraries	55
Figure 46: Load Dataset.....	56
Figure 47: Text Pre-Processing & Cleaning Code.....	56
Figure 48: Test & Train Split.....	57
Figure 49: Baseline SVM Model Training	57
Figure 50: Baseline SVM Model Evaluation Results	60
Figure 51: Confusion Matrix (Baseline SVM)	60
Figure 52: GridSearchCV Hyperparameter Implementation.....	62
Figure 53: Tuned SVM Model Evaluation Results.....	63
Figure 54: Grouped Bar Chart Visualization of Baseline vs Tuned SVM Model	64
Figure 55: Importing Libraries.....	65
Figure 56: Load and Read Dataset.....	66
Figure 57: Train and Test Split	66
Figure 58: Tokenization and Padding Code.....	67
Figure 59: Baseline LSTM Model Development.....	68
Figure 60: Baseline LSTM Model Training	69
Figure 61: Baseline LSTM Training Process.....	69
Figure 62: Baseline LSTM Model Evaluation.....	70
Figure 63: Baseline LSTM Model Evaluation Results	71
Figure 64: RandomizedSearchCV Hyperparameter Setup	72
Figure 65: Hyperparameter Search Space.....	72
Figure 66: RandomizedSearchCV Hyperparameter Tuning Execution.....	73

Figure 67: Hyperparameter Tuning Process	73
Figure 68: Best Hyperparameter Tuning Parameters and Accuracy Code	74
Figure 69: Best Hyperparameter Tuning Configuration and Accuracy Output.....	74
Figure 70: Tuned LSTM Model Evaluation Results.....	75
Figure 71: Tuned LSTM Model Evaluation Results Output	76
Figure 72: Baseline vs Tuned LSTM Comparison Code.....	77
Figure 73: Baseline vs Tuned LSTM Comparison Output	77
Figure 74: Importing Necessary Libraries	78
Figure 75: Load and Read Dataset.....	79
Figure 76: Dataset Load Confirmation and Preview	79
Figure 77: Train and Test Split	80
Figure 78: Train and Test Split Overview	80
Figure 79: TF-IDF Vectorization Configuration	81
Figure 80: TF-IDF Vectorization Fitting to Training Data.....	81
Figure 81: TF-IDF Vectorization Fitting Completion Preview	82
Figure 82: Logistic Regression Model Configuration & Parameters	82
Figure 83: Baseline Logistic Regression Model Training	83
Figure 84: Baseline Logistic Regression Model Training Confirmation.....	83
Figure 85: Most Influential Features Coefficients	84
Figure 86: Horizontal Bar Charts of Most Influential Features (Positive & Negative).....	84
Figure 87: Baseline Logistic Regression Model Results Code.....	85
Figure 88: Baseline Logistic Regression Model Evaluation Results.....	86
Figure 89: Baseline Logistic Regression Model Classification Report	86
Figure 90: Confusion Matrix (Baseline Logistic Regression)	87
Figure 91: RandomizedSearchCV Libraries Import	88
Figure 92: Hyperparamater Search Space Selection.....	88
Figure 93: Hyperparamater Search Space Selection Output.....	89
Figure 94: RandomizedSearchCV Hyperparameter Tuning Execution.....	90
Figure 95: RandomizedSearchCV Hyperparameter Tuning Process & Completion.....	90
Figure 96: Best RandomizedSearchCV Hyperparameter Combination Code	91
Figure 97: Best RandomizedSearchCV Hyperparameter Combination Output	91
Figure 98: Horizontal Bar Chart of Best Hyperparameter Combinations	92
Figure 99: Tuned Logistic Regression Model Results Code	93
Figure 100: Tuned Logistic Regression Model Evaluation & Classification Report	93

Figure 101: Confusion Matrix (Tuned Logistic Regression).....	94
Figure 102: Baseline vs Tuned Logistic Regression Model Comparison Code	95
Figure 103: Baseline vs Tuned Logistic Regression Model Comparison Output	95
Figure 104: Grouped Bar Chart Visualization of Baseline vs Tuned Logistic Regression Model	96
Figure 105: Libraries Import.....	97
Figure 106: Complete Baseline Random Forest Model Build.....	98
Figure 107: Train and Test Dataset Split	98
Figure 108: TF-IDF Vectorisation Implementation.....	99
Figure 109: Baseline Random Forest Model Development.....	100
Figure 110: Baseline Random Forest Model Evaluation Results	102
Figure 111: Confusion Matrix (Baseline Random Forest Model)	103
Figure 112: GridSearchCV Hyperparameter Tuning Execution Code	104
Figure 113: Tuned Random Forest Model Evaluation Results and Classification Report	108
Figure 114: Grouped Bar Chart Visualization of Baseline vs Tuned Random Forest Model	109
Figure 115: Support Vector Machine using LinearSVC after tuning (Sarvein)	110
Figure 116: Logistic Regression using TF-IDF Vectorization after Tuning (Rama)	111
Figure 117: LSTM – Irfan (base model).....	111
Figure 118: Random Forest - Kai Wen (Base Model).....	112

List of Tables

Table 1 : Stemming Comparison Table	20
Table 2: POS Tagging Methods Comparison	25
Table 3: POS Tagging Abbreviations and Full Form	25
Table 4: Evaluation Measures.....	58
Table 5: Hyperparameter Selection (SVM)	61
Table 6: Top 5 Positive Coefficient Features	84
Table 7: Top 5 Negative Coefficient Features	85
Table 8: Hyperparameter Selection (Logistic Regression)	89
Table 9: Best RandomizedSearchCV Hyperparameter Combination.....	91
Table 10: TF-IDF Parameter, Value and Justification Table.....	99
Table 11: Baseline Random Forest Model Parameters.....	100
Table 12: Evaluation Metric & Score Breakdown.....	102
Table 13: Confusion Matrix Per-Class Breakdown Table	103
Table 14: GridSearchCV Configuration Settings	105
Table 15: Hyperparameter Values Searched Table	106
Table 16: GridSearchCV Results Table.....	106
Table 17: Tuned Random Forest Model Evaluation Results Overview	108
Table 18: Tuned Random Forest Model Per-Class Results Overview	109

1.0 Introduction

1.1 Background of Text Analytics

Text analytics is the process of analyzing unstructured text data to extract meaningful information and insights. Unlike structured data, text data such as reviews, comments, and social media posts does not follow a fixed format, making it more challenging to analyze. Text analytics applies techniques from natural language processing (NLP) and machine learning to convert text into a structured form that can be processed by models. Common steps include tokenization, where text is broken into smaller units, and feature extraction, where text is transformed into numerical representations. One important application of text analytics is sentiment analysis, which is used to identify whether a piece of text expresses a positive, negative, or neutral opinion. This is widely used in areas such as customer feedback analysis, product reviews, and decision-making processes.

1.2 Assignment Overview

In this assignment, text analytics techniques are applied to a dataset of reviews to perform sentiment classification. In Part 1, the focus is on preparing the text data for analysis. This includes preprocessing steps such as tokenization and converting the text into numerical form so that it can be used by machine learning models. Different approaches to text representation are explored to understand how text can be transformed into meaningful features. In Part 2, supervised learning methods are used to build classification models that can predict whether a review is positive or negative. A Long Short-Term Memory (LSTM) model is implemented as it is suitable for handling sequential text data and capturing contextual information. Hyperparameter tuning is then performed using RandomizedSearchCV to improve the model's performance by testing different parameter combinations. Finally, the models are evaluated using multiple metrics such as accuracy, precision, recall, and F1-score, and a comparison is made between the baseline and tuned models to determine which approach performs better.

2.0 Part A: Text Pre-processing and Basic NLP Techniques

2.1 (Q1) Tokenization and Stop Words Filtering

2.1.1 Word Tokenization using Split Function, Regular Expressions and NLTK

Tokenization is the process of breaking down a text corpus into smaller units called *tokens*, typically words, which serve as the fundamental elements for text analysis.

```
# import library
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string

# Load data
with open('../Part_A/data/Data_1.txt', 'r', encoding='utf-8') as file:
    text_data = file.read()

# Tokenization Methods
tokens_split = text_data.split() #split
tokens_re = re.findall(r'\w+', text_data) #regex
tokens_nltk = word_tokenize(text_data) #nltk

# range to display (indices 15 to 25)
start, end = 15, 25

print(f"{'Index':<8} | {'Split()':<15} | {'Regex':<15} | {'NLTK':<15}")
print("-" * 60)

for i in range(start, end):
    # Handling potential index out of bounds
    s = tokens_split[i] if i < len(tokens_split) else ""
    r = tokens_re[i] if i < len(tokens_re) else ""
    n = tokens_nltk[i] if i < len(tokens_nltk) else ""
    print(f"{i:<8} | {s:<15} | {r:<15} | {n:<15}")
```

Figure 1: Importing Libraries

In this study, three different tokenization approaches were implemented:

- The `split()` function divides text based on whitespace. It is simple and computationally efficient but does not handle punctuation effectively.
- Regular expressions allow pattern-based token extraction. In this implementation, the pattern is used to extract only alphanumeric word tokens, effectively removing punctuation.
- The Natural Language Toolkit (NLTK) tokenizer applies pre-trained linguistic rules to segment text more accurately. It is capable of handling punctuation, contractions, and special cases.

Index	Split()	Regex	NLTK
15	basic	basic	In
16	classification	classification	basic
17	tasks,	tasks	classification
18	each	each	tasks
19	input	input	,
20	is	is	each
21	considered	considered	input
22	in	in	is
23	isolation	isolation	considered
24	from	from	in

Figure 2: Tokenization Approaches Output

The outputs generated from these three methods demonstrate noticeable differences in how tokens are formed, particularly in handling punctuation and sentence structure.

2.1.2 Comparison of Tokenization Methods

Based on the obtained outputs, the three tokenization techniques show different strengths and limitations.

The `split()` function is the fastest and simplest method, but it fails to separate punctuation from words. For example, tokens such as “input.” remain attached to punctuation, which may affect further analysis.

Regular expression tokenization improves this by extracting only word characters, thereby removing punctuation. However, it lacks linguistic awareness and may struggle with more complex text structures such as contractions like “don’t” or abbreviations.

In contrast, NLTK’s `word_tokenize` provides the most accurate and linguistically informed tokenization. It correctly separates punctuation from words and handles complex cases such as sentence boundaries and contractions. This makes it more suitable for real-world text analytics tasks.

Therefore, NLTK tokenization is considered the most suitable approach due to its balance between accuracy and robustness, despite having slightly higher computational overhead.

2.1.3 Stop Words and Punctuation Removal

After tokenization, removal of a stop words and punctuation is performed. Stop words refer to commonly used words such as “the”, “is”, and “and”, which carry minimal semantic meaning. While punctuation marks (e.g., “.”, “,”, “!”) are also removed as they do not contribute significantly to most text analysis tasks.

```

# 2. Filtering
stop_words = set(stopwords.words('english'))
punctuations = set(string.punctuation)

# unique stop words in the corpus
found_stop_words = sorted(list(set([w.lower() for w in tokens_nltk if w.lower() in stop_words])))

# Filtered tokens
filtered_tokens = [w for w in tokens_nltk if w.lower() not in stop_words and w not in punctuations]

# --- Table Output ---
# 1. Headers
header1 = "Unique Stop Words Found"
header2 = "Filtered Tokens"

print(f"{'Index':<4} | {header1:<24} | {header2}")
print("-" * 50)

# 2. Determine the maximum length to iterate through both lists
max_len = max(len(found_stop_words), len(filtered_tokens[:50]))
# 3. Print side-by-side vertically
for i in range(max_len):
    # Get stop word if it exists at this index
    stop_word = found_stop_words[i] if i < len(found_stop_words) else ""
    # Get filtered token if it exists at this index
    filtered_token = filtered_tokens[i] if i < 20 else ""
    # Only print if there is content in at least one column
    if stop_word or filtered_token:
        print(f"{'Index':<4} | {stop_word:<22} | {filtered_token}")

```

Figure 3: Stop Words and Punctuation Removal Code

In this implementation, stop words were obtained using the NLTK stop words corpus. Punctuation symbols were defined using Python's `string.punctuation` and tokens were filtered by removing both stop words and punctuation from the tokenized output.

Index	Unique Stop Words Found	Filtered Tokens
1	a	Classification
2	all	task
3	and	choosing
4	are	correct
5	be	class
6	each	label
7	for	given
8	from	input
9	has	basic
10	in	classification
11	is	tasks
12	not	input
13	of	considered
14	other	isolation
15	the	inputs
16		set
17		labels
18		defined
19		advance
20		basic

Figure 4: Stop Words and Punctuation Removal Output

Additionally, a list of unique stop words found within the corpus was extracted to provide insight into the dataset composition.

2.1.4 Importance of Removing Stop Words and Punctuation

Removing stop words and punctuation is an essential step in text preprocessing as it helps improve the overall quality and efficiency of text analysis. Stop words such as “the”, “is”, and “and” occur frequently but carry minimal semantic meaning, and their presence can introduce noise that obscures important keywords. By eliminating these words, the model can focus on more meaningful terms that contribute to better insights. Additionally, removing unnecessary tokens reduces the dimensionality of the dataset, which improves computational efficiency and speeds up processing. Punctuation marks can also create inconsistencies in token representation, such as treating “tasks” and “tasks.” as different tokens, which may negatively impact analysis accuracy. Therefore, filtering both stop words and punctuation ensures cleaner, more consistent data and enhances the performance of downstream text analytics tasks.

2.2 (Q2) Word Stemming

2.2.1 Importance of Stemming in Text Analytics

Stemming refers to the process of reducing words to their root or base form by removing common suffixes. For instance, words such as “classification,” “classified,” and “classifying” may all be reduced to a shared stem like “class” or “classif.” This technique is widely used as a pre-processing step in text analytics and natural language processing (NLP) to improve data consistency and analytical efficiency.

The importance of stemming can be understood through several key aspects:

1. Vocabulary and Dimensionality Reduction

Text datasets often contain multiple variations of the same word (e.g., “task,” “tasks,” “tasked”), which are treated as separate features if left unprocessed. Stemming consolidates these variations into a single representation, thereby reducing the size of the vocabulary and the dimensionality of the feature space. This simplification enhances computational efficiency and can help mitigate overfitting in machine learning models.

2. Improved Information Retrieval

Stemming enhances the effectiveness of search and retrieval systems by allowing different forms of a word to be treated equivalently. For example, a query for “classify” can retrieve documents containing “classification” or “classified.” As a result, stemming improves recall

by increasing the likelihood of retrieving relevant documents, even when there are variations in word forms.

3. Better Feature Normalisation for Machine Learning

By standardising word forms, stemming provides more consistent input for machine learning algorithms such as Naive Bayes, Support Vector Machines (SVM), and TF-IDF vectorisation. This allows models to learn more effectively from the data, as information is not fragmented across multiple variants of the same term. This is particularly beneficial when working with smaller datasets, where data sparsity may otherwise limit model performance.

2.2.2 Implementation and Output of Regular Expression, Porter Stemmer and Lancaster Stemmer

The following Python code loads the text corpus from Data_1.txt, tokenises it, and applies three stemming methods: Regular Expression Stemmer, Porter Stemmer, and Lancaster Stemmer. The output is displayed in a side-by-side comparison table.

Step1: The text corpus is loaded from Data_1.txt using Python's built-in file reader.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, LancasterStemmer, RegexpStemmer

# 1. Load Data_1.txt
with open("../Part_A/data/Data_1.txt", "r", encoding="utf-8") as f:
    data_1 = f.read()

print("Data_1.txt:")
print(data_1)
```

Figure 5: Read the Dataset

Output:

```
Data_1.txt:
Classification is the task of choosing the correct class label for a given input. In basic
classification tasks, each input is considered in isolation from all other inputs, and the set of labels is defined in advance. The basic classification ta
```

Figure 6: Dataset Output Confirmation

Step 2 : word_tokenize() from NLTK is used to break the raw text into individual word tokens.

Step 3: RegexpStemmer is configured with a custom suffix pattern 'ing\$|s\$|ed\$|able\$|ion\$|ment\$' and a minimum stem length of 4 characters. This means it

will only strip a suffix if the resulting stem has at least 4 characters, preventing over-truncation of short words.

```
# 2. Initialize stemmers
porter = PorterStemmer()
lancaster = LancasterStemmer()

# Implementing rule set for Regular Expression Stemmer
regex = RegexpStemmer('ing$|s$|ed$|able$|ion$|ment$', min=4)

# 3. Tokenize the Data_1.txt sentences into each word to undergo stemming
words = [w for w in word_tokenize(data_1)]

print("Data_1.txt - Word Tokenization:")
print(words)
```

Figure 7: Initialization of Stemmers & Word Tokenization

```
Data_1.txt - Word Tokenization:
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', 'classifica
```

Figure 8: Word Tokenization Output

Step 4: `sorted(list(set(...)))` removes duplicate tokens and sorts them alphabetically, making the comparison table easier to read.

```
# 4. Sort words & Remove redundant words from output
words = sorted(list(set(words)))

print("Data_1.txt - SORTED Word Tokenization:")
print(words)
```

Figure 9: Sorting Unique Words

Output:

```
Data_1.txt - SORTED Word Tokenization:
['.', ':', 'Classification', 'For', 'In', 'The', 'a', 'advance', 'all', 'and', 'are', 'assigned', 'basic', 'be', 'choosing', 'class', 'classification'
```

Figure 10: Sorted Unique Words Output

Step 5: `PorterStemmer()` and `LancasterStemmer()` are used with their default settings as provided by NLTK.

```
# 5. Comparison table
print("\n 2.2 Demonstrate word stemming using Regular Expression, Porter Stemmer and Lancaster Stemmer and report the output. \n")
print(f"{'Original':<20} | {'Regular Expression':<20} | {'Porter':<20} | {'Lancaster':<20}")
print("-" * 90)
for each_word in words:
    # Regular Expression Stemming
    r_stem = regex.stem(each_word)
    # Porter Stemming
    p_stem = porter.stem(each_word)
    # Lancaster Stemming
    l_stem = lancaster.stem(each_word)
    # Print table
    print(f"{'each_word':<20} | {'r_stem':<20} | {'p_stem':<20} | {'l_stem':<20}")
```

Figure 11: Stemmer Implementation and Result Table Code

A formatted table is printed, showing the original word alongside the result from each of the three stemmers for direct comparison.

Output:

2.2 Demonstrate word stemming using Regular Expression, Porter Stemmer and Lancaster Stemmer and report the output.			
Original	Regular Expression	Porter	Lancaster
,	,	,	,
.	.	.	.
;	;	;	;
Classification	Classificat	classif	class
For	For	for	for
In	In	in	in
The	The	the	the
a	a	a	a
advance	advance	advanc	adv
all	all	all	al
and	and	and	and
are	are	are	ar
assigned	assign	assign	assign
basic	basic	basic	bas
be	be	be	be
choosing	choos	choos	choos
class	clas	class	class
classification	classificat	classif	class
classified	classifi	classifi	class
considered	consider	consid	consid
correct	correct	correct	correct
defined	defin	defin	defin
each	each	each	each
example	example	exampl	exampl
for	for	for	for

Figure 12: Stemmer Implementation and Result Table Output

Code Explanation

- The text corpus is loaded from Data_1.txt using Python's built-in file reader.
- `word_tokenize()` from NLTK is used to break the raw text into individual word tokens.
- `sorted(list(set(...)))` removes duplicate tokens and sorts them alphabetically, making the comparison table easier to read.
- `RegexStemmer` is configured with a custom suffix pattern `'ing$|s$|ed$|able$|ion$|ment$'` and a minimum stem length of 4 characters. This means it will only strip a suffix if the resulting stem has at least 4 characters, preventing over-truncation of short words.

- PorterStemmer() and LancasterStemmer() are used with their default settings as provided by NLTK.
- A formatted table is printed, showing the original word alongside the result from each of the three stemmers for direct comparison.

2.2.3 Differences among Regular Expression stemmer, Porter Stemmer and Lancaster Stemmer

Based on the output observed in the comparison table above, the three stemmers shows distinct behaviours in terms of their aggressiveness, consistency, accuracy, and readability. The following analysis examines each stemmer in detail.

2.3.3.1 Regular Expression (Regex) Stemmer

The Regex Stemmer operates by matching and removing a user-defined list of suffixes from the end of each word. In this implementation, the pattern 'ing\$|s\$|ed\$|able\$|ion\$|ment\$' was used, meaning the stemmer strips any of those six suffix types, provided the resulting stem is at least 4 characters long.

Key observations from the output:

- The Regex Stemmer is the most conservative (least aggressive) of the three. Words without a matching suffix are returned unchanged (e.g., "advance", "example", "given" remain as-is).
- It correctly strips the -s suffix from plural nouns: "inputs" -> "input", "labels" -> "label", "tasks" -> "task".
- It strips -ion from words like "Classification" -> "Classificat" and "isolation" -> "isolat", but does not further reduce the stem.
- The Regex Stemmer preserves capitalisation, which means "Classification" is stemmed to "Classificat" while "classification" is stemmed to "classificat". This is a limitation, as case-sensitivity is not handled automatically.
- Because the rules are entirely user-defined, the stemmer's behaviour is predictable but limited; it cannot handle irregular word forms or complex morphology beyond the specified patterns.

2.3.3.2 Porter Stemmer

The Porter Stemmer is one of the most widely used stemming algorithms in NLP. It applies a sequence of five rule-based phases to progressively strip suffixes and reduce words to their root form. It is designed to be a good balance between aggressiveness and readability.

Key observations from the output:

- The Porter Stemmer is more thorough than the Regex Stemmer. It reduces words beyond simple suffix removal, e.g., "advance" -> "advanc", "example" -> "exempl", and "sequence" -> "sequenc".
- It correctly normalises case: "Classification" (capitalised) and "classification" (lowercase) both stem to "classif", demonstrating case normalisation.
- Compound stemming results are mostly readable and semantically meaningful. For example, "interesting" -> "interest" and "jointly" -> "jointli" are recognisable as roots.
- Occasionally produces non-words as stems (e.g., "has" -> "ha"), which is a known limitation of rule-based stemming. These are linguistically incorrect but still achieve term normalisation.
- It handles the merging of related words effectively: "classification", "classified", and "classifying" all converge to "classif", ensuring these conceptually identical words are treated as one feature.

2.3.3.3 Lancaster Stemmer

The Lancaster Stemmer (also called the Paice/Husk stemmer) is the most aggressive of the three algorithms. It uses an iterative approach driven by a large lookup table of rules and continues applying rules until no more apply, resulting in much shorter stems.

Key observations from the output:

- The Lancaster Stemmer produces the shortest stems. For example, "advance" -> "adv", "instance" -> "inst", "isolation" -> "isol", and "variants" -> "vary".
- Notably, it can change the stem form entirely: "variants" -> "vary" demonstrates that the Lancaster Stemmer sometimes applies phonetic transformations, not just suffix removal.

- It over-stems aggressively in several cases: "all" -> "al", "are" -> "ar", "other" -> "oth", and "number" -> "numb". These shortened forms are often not recognisable as English words.
- One advantage is maximum normalisation: "Classification", "classification", "classified" all reduce to "class", achieving full convergence. This is the strongest term grouping of the three methods.
- The trade-off is loss of semantic clarity. "basic" -> "bas" and "jointly" -> "joint" are less informative than the roots produced by Porter or Regex stemmers.

2.3.3.4 Summary of differences

Aspect	Regex Stemmer	Porter Stemmer	Lancaster Stemmer
Approach	User-defined pattern rules (suffix list)	Rule-based linguistic algorithm (5 phases)	Iterative table-driven algorithm (very aggressive)
Aggressiveness	Low to Medium (depends on rules)	Medium (balanced)	High (most aggressive)
Example: "variants"	"variant" (removes -s)	"variant" (removes -s)	"vary" (changes stem entirely)
Example: "advance"	"advance" (no suffix match)	"advanc" (removes final e)	"adv" (heavily truncates)
Example: "isolation"	"isolat" (removes -ion)	"isol" (removes -ation)	"isol" (same result)
Readability of stems	Highest (most recognisable)	Medium (usually readable)	Lowest (often truncated)
Speed	Fastest (simple regex match)	Moderate	Fastest of rule-based stemmers
Customisability	Fully customisable by developer	Fixed algorithm, not customisable	Fixed algorithm, not customisable
Best used for	Domain-specific or limited vocabulary	General NLP tasks, balanced accuracy	When maximum recall is needed

Table 1 : Stemming Comparison Table

Conclusion

The choice of stemmer should be guided by the requirements of the task. The Regex Stemmer offers maximum developer control and transparency but requires manual rule engineering. The Porter Stemmer provides a well-tested, balanced approach suitable for most general NLP tasks such as document classification and information retrieval. The Lancaster Stemmer is preferred in scenarios where maximum term consolidation is needed and readability of stems is less important, such as in high-recall search systems or when dealing with a very large, varied vocabulary. For the given corpus, the Porter Stemmer offers the best balance between normalisation effectiveness and stem readability.

2.3 (Q3) Parts of Speech Tagging and Syntactic Analysis

2.3.1 POS Tagging Demonstration

2.3.1.1 POS Tagging using NLTK

3.1.1 NLTK POS Tagger

```
print("\n1.1 NLTK POS Tagger\n")

# Tokenize the sentence into individual words/tokens
tokens = nltk.word_tokenize(sentence)
print(f"\nTokenized Words ({len(tokens)} tokens):")
print(tokens)

# POS tagging
nltk_pos = nltk.pos_tag(tokens)

print("\n\nNLTK POS Tagging Output:")
print(f"{'Word':<15} {'POS Tag':<10} {'Description':}")

# Penn Treebank POS tag set
pos_meanings = {
    'DT': 'Determiner',
    'JJ': 'Adjective',
    'NN': 'Noun, singular',
    'NNS': 'Noun, plural',
    'VBD': 'Verb, past tense',
    'VB': 'Verb, base form',
    'VBG': 'Verb, gerund/present participle',
    'VBP': 'Verb, non-3rd person singular present',
    'IN': 'Preposition/subordinating conjunction',
    'CC': 'Coordinating conjunction',
    'RB': 'Adverb',
    '': 'Punctuation',
    ',': 'Comma',
    'PRP': 'Personal pronoun'
}

# Iterate through each word-tag pair and display with description
for word, tag in nltk_pos:
    # Get the meaning of the tag, default to the tag itself if not in dictionary
    meaning = pos_meanings.get(tag, tag)
    print(f"{'word':<15} {'tag':<10} {'meaning':}")

# Display the complete list of tuples for reference
print("\n\nComplete Output (List of Tuples):")
print(nltk_pos)
```

The sentence "The big black dog barked at the white cat and chased away." was divided into 13 words, and the parts of speech were correctly assigned by the NLTK library. The words "big," "black," and "white" were correctly assigned the part of speech JJ, "dog" and "cat" were assigned the part of speech NN, "barked" and "chased" were assigned the part of speech VBD, "at" was assigned the part of speech IN, "and" was assigned the part of speech CC, "away" was assigned the part of speech RB, and finally, "The" and "the" were assigned the part of speech DT.

2.3.1.2 POS Tagging using TextBlob

3.1.2 TextBlob POS Tagger

```
# Section 3.1.2: Part-of-Speech Tagging using TextBlob
print("\n\n3.1.2 TEXTBLOB POS TAGGER\n\n")

# Create a TextBlob object from the sentence
# TextBlob automatically tokenizes and tags the text
blob = TextBlob(sentence)

# Access the POS tags using the .tags property
textblob_pos = blob.tags

# Display formatted POS tagging results
print("TextBlob POS Tagging Output:")
print(f"{'Word':<15} {'POS Tag':<10} {'Description'}")

# Iterate through each word-tag pair and display with description
for word, tag in textblob_pos:
    # Get the meaning of the tag from the previously defined dictionary
    meaning = pos_meanings.get(tag, 'Unknown')
    print(f"{'word':<15} {'tag':<10} {'meaning'}")

# Display the complete list of tuples for reference
print("\n\nComplete Output:")
print(list(textblob_pos))
```

Figure 15: POS Tagging using TextBlob Code

This cell creates a TextBlob object from the sentence provided, which automatically tokenizes and tags each word, and then accesses the tags through the .tags attribute and displays each word and its tag and description using the same Penn Treebank tag set defined previously.

3.1.2 TEXTBLOB POS TAGGER

TextBlob POS Tagging Output:

Word	POS Tag	Description
The	DT	Determiner
big	JJ	Adjective
black	JJ	Adjective
dog	NN	Noun, singular
barked	VBD	Verb, past tense
at	IN	Preposition/subordinating conjunction
the	DT	Determiner
white	JJ	Adjective
cat	NN	Noun, singular
and	CC	Coordinating conjunction
chased	VBD	Verb, past tense
away	RB	Adverb

Complete Output:

```
[('The', 'DT'), ('big', 'JJ'), ('black', 'JJ'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'JJ'), ('cat', 'NN'), ('and', 'CC'), ('chased', 'VBD'), ('away', 'RB')]
```

Figure 16: POS Tagging Output (TextBlob)

TextBlob produced the same POS tags as NLTK for all the words in the sentence. It correctly identified the determiners "the" and "this," the adjectives "big" and "good," the noun "dog," the verb "runs," the preposition "around," the coordinating conjunction "and," and the adverb

"very." The only difference from the NLTK tags is the exclusion of the punctuation mark "." from the output by TextBlob, while NLTK included it as a separate word.

2.3.1.3 POS Tagging using Regular Expression Tagger

3.1.3 Regular Expression Tagger

```
# Section 3.1.3: Part-of-Speech Tagging using Regular Expressions
print("\n3.1.3 REGULAR EXPRESSION TAGGER\n")

# Define pattern-tag pairs for rule-based POS tagging
patterns = [
    (r'.*ing$', 'VBG'),      # gerunds
    (r'.*ed$', 'VBD'),      # simple past
    (r'.*es$', 'VBZ'),      # 3rd singular present
    (r'.*ould$', 'MD'),     # modals
    (r'.*\''s$', 'NN$'),     # possessive nouns
    (r'.*s$', 'NNS'),       # plural nouns
    (r'^[0-9]+([0-9]+)?$', 'CD'), # cardinal number
    (r'^(The|the|a|an)$', 'DT'), # determiners
    (r'^(and|or|but)$', 'CC'), # coordinating conjunctions
    (r'^(at|in|on|to|from)$', 'IN'), # prepositions
    (r'.*ly$', 'RB'),       # adverbs
    (r'.*$', 'NN')
]

# Create a Regular Expression Tagger with the defined patterns
regex_tagger = nltk.RegexpTagger(patterns)

# Apply the regex tagger to the previously tokenized words
regex_pos = regex_tagger.tag(tokens)

# Display formatted POS tagging results
print("Regular Expression POS Tagging Output:")
print(f"{'Word':<15} {'POS Tag':<10} {'Description'}")

# Iterate through each word-tag pair and display with description
for word, tag in regex_pos:
    # Get the meaning of the tag from the dictionary
    meaning = pos_meanings.get(tag, 'Unknown')
    print(f"{'word':<15} {'tag':<10} {'meaning'}")

# Display the complete list of tuples for reference
print("\n\nComplete Output:")
print(regex_pos)
```

Figure 17: POS Tagging using Regular Expression Tagger Code

This cell sets a series of regex patterns that map word endings and structures to a POS tag. A `RegexpTagger` is then built based on this set of patterns, which is used to tag each word based on which pattern matches the word. This is done on the previously tokenized words.

3.1.3 REGULAR EXPRESSION TAGGER

Regular Expression POS Tagging Output:

Word	POS Tag	Description
The	DT	Determiner
big	NN	Noun, singular
black	NN	Noun, singular
dog	NN	Noun, singular
barked	VBD	Verb, past tense
at	IN	Preposition/subordinating conjunction
the	DT	Determiner
white	NN	Noun, singular
cat	NN	Noun, singular
and	CC	Coordinating conjunction
chased	VBD	Verb, past tense
away	NN	Noun, singular
.	NN	Noun, singular

Complete Output:

```
[('The', 'DT'), ('big', 'NN'), ('black', 'NN'), ('dog', 'NN'), ('barked', 'VBD'), ('at', 'IN'), ('the', 'DT'), ('white', 'NN'), ('cat', 'NN'), ('and', 'CC'), ('chased', 'VBD'), ('away', 'NN'), ('.', 'NN')]
```

Figure 18: POS Tagging Output (Regular Expression Tagger)

The `RegexpTagger` successfully identified the determiners ("The", "the"), the past tense verbs ("barked", "chased"), the preposition ("at"), and the coordinating conjunction ("and"). However, it incorrectly identified "big", "black", and "white" as nouns (NN) rather than adjectives (JJ), and incorrectly identified "away" as a noun (NN) rather than an adverb (RB),

and also incorrectly identified the punctuation mark "." as a noun (NN). This demonstrates the main drawback of the RegexpTagger, which is that it only relies on the patterns of the words, rather than any understanding of them, and is thus not as accurate as the NLTK and TextBlob models, which use statistical models to understand the grammatical role of each word.

2.3.2 Comparison of POS Tagging Methods

Comparison Table

Table 2: POS Tagging Methods Comparison

Word	NLTK	TextBlob	RegEx	Status
The	DT	DT	DT	All match
big	JJ	JJ	NN	Regex differs
black	JJ	JJ	NN	Regex differs
dog	NN	NN	NN	All match
barked	VBD	VBD	VBD	All match
at	IN	IN	IN	All match
the	DT	DT	DT	All match
white	JJ	JJ	NN	Regex differs
cat	NN	NN	NN	All match
and	CC	CC	CC	All match
chased	VBD	VBD	VBD	All match
away	RB	RB	RB	All match

Table 3: POS Tagging Abbreviations and Full Form

Abbreviation	Full Form
DT	Determiner
JJ	Adjective
NN	Noun, singular or mass
VBD	Verb, past tense
IN	Preposition or subordinating conjunction
CC	Coordinating conjunction
RB	Adverb

NLTK and TextBlob both use statistical machine learning models trained on the Penn Treebank corpus (1M+ words), making them context-aware and highly accurate at 95-97%. They analyze surrounding words to determine correct tags, successfully identifying all parts of speech including adjectives like 'big', 'black', and 'white' as JJ with 100% accuracy on the test sentence. TextBlob is essentially a simplified wrapper around NLTK's tagger, offering the same accuracy with an easier API for beginners, while NLTK is preferred for production systems. Both taggers handle ambiguous words well by recognizing context-dependent meanings, making them suitable for serious NLP applications.

In contrast, the Regular Expression tagger uses rule-based pattern matching with predefined suffix/prefix rules and no training data, achieving only 76.9% accuracy. It is not context-aware and relies solely on word patterns, causing it to fail on simple adjectives like 'big', 'black', and 'white', incorrectly tagging them as nouns (NN) instead of adjectives (JJ) because these words lack distinctive suffixes. While the RegEx tagger is fast, transparent, and customizable for specific domains, its 20-40% typical accuracy demonstrates that statistical taggers significantly outperform rule-based approaches for general text due to their ability to learn from data and understand context rather than relying on rigid pattern-matching rules.

2.3.3 Parse Tree Generation

Method 1: Chunk Parser

```

3.3 Draw Parse Trees

# QUESTION 3: PARSE TREES

# METHOD 1: Chunking using Chunk Parser
print("\n\nMETHOD 1: CHUNK PARSER\n")

# Define grammar rules for chunking using regular expressions
chunk_grammar = r"""
NP: {<DT>*<JJ>*<NN.*>+}
VP: {<VB.*>*<RB>?}
VP: {<VB.*>*<IN>*<NP>}
PP: {<IN>*<NP>}
"""

# Create a chunk parser with the defined grammar rules
chunk_parser = RegexpParser(chunk_grammar)

# Parse the POS-tagged tokens to identify and group phrases
# Returns a tree structure with labeled chunks
chunked_tree = chunk_parser.parse(nltk_pos)

# Display the tree structure in text format
print("Parse Tree Output:")
print(chunked_tree)

# Display with svglint for beautiful SVG visualization
print("\n\nSVG Visualization (svglint):")
display(chunked_tree)

```

Figure 19: Chunk Parser Code

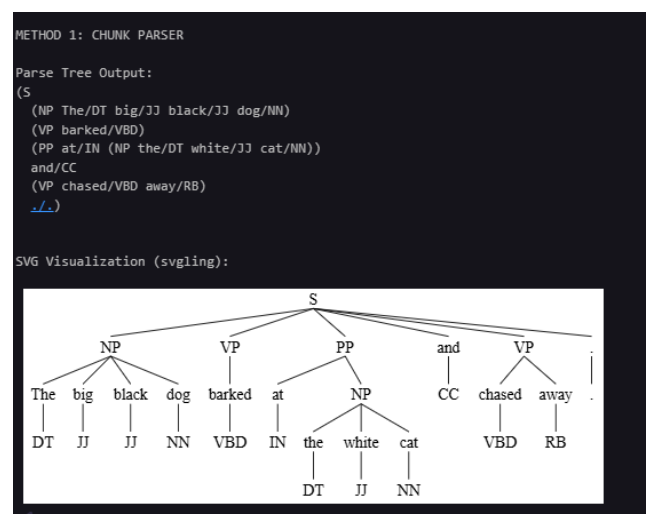


Figure 20: Parse Tree Output (Chunk Parser)

This cell employs regular expression rules to construct a chunk grammar encompassing four phrase types: noun phrases (NP), verb phrases (VP), prepositional phrases (PP), and their combinations. Subsequently, these grammar rules are utilized to instantiate a `RegexpParser` object. This object is then used to process the NLTK POS-tagged tokens. This creates a tree structure that is available both in text and SVG formats.

The parse tree was able to analyse the grammatical structure of the sentence provided. The sentence was broken down into its parts. For instance, the noun phrase "The big black dog" was identified and then broken down into its parts. The classification of the word "barked" as a verb phrase (VP) was correct. The phrase "At the white cat" was correctly identified as a prepositional phrase (PP), given its constituents: a preposition and a noun phrase "the white cat." Furthermore, "And" was correctly identified as a single coordinating conjunction (CC), and "chased away" was correctly identified as a second verb phrase (VP). The SVG graphic also accurately represents the hierarchical structure of the sentence, illustrating the breakdown of the sentence (S) into its constituent parts, with the prepositional phrase functioning as a sub-tree that includes a nested noun phrase.

Method 2: CONTEXT-FREE GRAMMAR PARSER

```
# METHOD 2: Parsing using Context-Free Grammar (CFG)
print("\n\nMETHOD 2: CONTEXT-FREE GRAMMAR PARSER\n\n")

with open("../Part_A/data/Data_2.txt", "r", encoding="utf-8") as f:
    sentence = f.read().strip()

tokens = nltk.word_tokenize(sentence)
nltk_pos = nltk.pos_tag(tokens)

# Define a context-free grammar with production rules
cfg_grammar = nltk.CFG.fromstring("""
S -> NP VP
S -> NP VP CC VP

NP -> DT ADJP NN
NP -> DT NN

ADJP -> JJ JJ
ADJP -> JJ

VP -> VBD PP
VP -> VBD RB

PP -> IN NP

DT -> 'The' | 'the'
JJ -> 'big' | 'black' | 'white'
NN -> 'dog' | 'cat'
VBD -> 'barked' | 'chased'
IN -> 'at'
CC -> 'and'
RB -> 'away'
""")

# Remove punctuation from tokens for cleaner parsing
parse_tokens = [word for word in tokens if word != '.']

# Create a chart parser with the defined CFG grammar
parser = nltk.ChartParser(cfg_grammar)

# Parse the tokens and generate all possible parse trees
parse_trees = list(parser.parse(parse_tokens))

# Display each parse tree with both text and visual formats
for i, tree in enumerate(parse_trees, 1):
    print(f"\n\nParse Tree Output:")
    print(tree)
    print("\n\nSVG Visualization (svgling):")
    display(tree)
```

Figure 21: Context-Free Grammar Parser Code

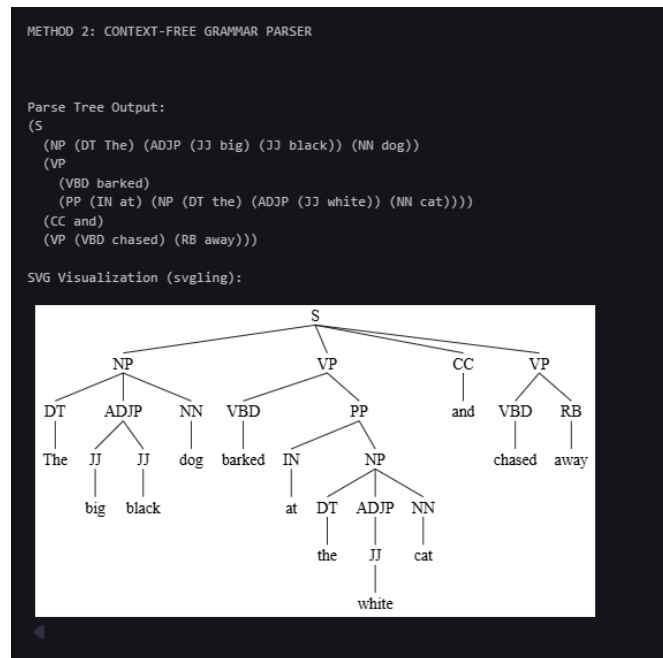


Figure 22: Parse Tree Output (Context-Free Grammar Parser)

This cell reads the sentence from Data_2.txt, tokenizes it, and performs POS tagging. It also creates a Context-Free Grammar (CFG) with specific rules for how words are grouped into phrases. The rules are for sentences (S), noun phrases (NP), adjective phrases (ADJP), verb phrases (VP), and prepositional phrases (PP), among other rules. It also contains terminal rules, which define how each word is mapped to a specific part of speech. A ChartParser is then constructed from the CFG, and it is applied to the tokens.

The parse tree constructed using the CFG parser was more detailed and hierarchically deeper than the parse tree constructed using Method 1. The noun phrase "The big black dog" was more deeply parsed, with "big" and "black" grouped into a separate adjective phrase (ADJP) within the NP. The verb phrase also contains a deeper nesting, with "barked" followed by "at the white cat," "white" again grouped into a separate ADJP within the inner NP. "And" is correctly parsed as a standalone CC, and "chased away" is correctly parsed as a VP, with "away" correctly parsed as an RB. The CFG parser constructed a more detailed and hierarchically deeper parse tree than the chunk parser, since it is based on strict grammar rules.

2.4 (Q4) Sentence Probability using Bigram Models

2.4.1 Unsmoothed Bigram Model

The bigram probability formula (MLE) is:

$$P(w_i | w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$$

Each consecutive word pair in the test sentence is evaluated as follows:

Bigram	Formula	Fraction	Probability
P(I <s>)	$C(<s>, I) / C(<s>)$	1 / 3	0.3333
P(read I)	$C(I, \text{read}) / C(I)$	1 / 1	1.0000
P(a read)	$C(\text{read}, a) / C(\text{read})$	3 / 3	1.0000
P(different a)	$C(a, \text{different}) / C(a)$	1 / 3	0.3333
P(book different)	$C(\text{different}, \text{book}) / C(\text{different})$	1 / 1	1.0000
P(by book)	$C(\text{book}, \text{by}) / C(\text{book})$	1 / 3	0.3333
P(Danielle by)	$C(\text{by}, \text{Danielle}) / C(\text{by})$	1 / 1	1.0000
P(</s> Danielle)	$C(\text{Danielle}, </s>) / C(\text{Danielle})$	1 / 1	1.0000

The sentence probability is the product of all conditional probabilities:

$$\begin{aligned}
 P(\text{sentence}) &= 1/3 \times 1 \times 1 \times 1/3 \times 1 \times 1/3 \times 1 \times 1 \\
 &= 1/27 \approx 0.03704
 \end{aligned}$$

2.4.2 Smoothed Bigram Model

Laplace smoothing adds 1 to every bigram count and adds V (vocabulary size) to every unigram denominator:

$$P(w_i | w_{i-1}) = (C(w_{i-1}, w_i) + 1) / (C(w_{i-1}) + V)$$

With $V = 10$, each conditional probability is adjusted as follows:

Bigram	Formula (count+1)/(prev+V)	Fraction	Probability
P(I <s>)	(1+1)/(3+10)	2/13	0.1538
P(read I)	(1+1)/(1+10)	2/11	0.1818
P(a read)	(3+1)/(3+10)	4/13	0.3077
P(different a)	(1+1)/(3+10)	2/13	0.1538
P(book different)	(1+1)/(1+10)	2/11	0.1818
P(by book)	(1+1)/(3+10)	2/13	0.1538
P(Danielle by)	(1+1)/(1+10)	2/11	0.1818
P(</s> Danielle)	(1+1)/(1+10)	2/11	0.1818

The sentence probability is:

$$P(\text{sentence}) = 2/13 \times 2/11 \times 4/13 \times 2/13 \times 2/11 \times 2/13 \times 2/11 \times 2/11$$

$$= 512 / 418,161,601 \approx 1.224 \times 10^{-6}$$

The smoothed probability is considerably lower than the unsmoothed result because probability mass has been redistributed from observed bigrams to all possible unseen bigrams. This is the intended trade-off where the model no longer assigns zero probability to unknown sequences.

2.4.3 Python Implementation of Bigram Models

Load the Data 3 file

Question 4

```
#1 Open the file and read everything inside it as one big string

with open("../Part_A/data/Data_3.txt", "r", encoding="utf-8") as f:
    text = f.read()

print(text)
```

✓ 0.0s

Training Corpus

```
<s> He read a book </s>
<s> I read a different book </s>
<s> He read a book by Danielle </s>
```

Calculate sentence probability for the following sentence

```
<s> I read a different book by Danielle </s>
```

Figure 23: Data Import

This cell opens Data_3.txt and reads its entire contents into the variable text as a single string, then prints it to verify the file was loaded correctly.

Pre-Process the Text Corpus

2. Step 2 — Preprocess (keep and-) What preprocessing we do (simple + safe)

We:

- split by lines (each line is one sentence)
- split each sentence by spaces (so and-stay as tokens)
- remove any empty pieces

```
# 2) Split into lines, keep only training sentences (lines starting with <s>)
lines = [line.strip() for line in text.splitlines() if line.strip().startswith('<s>')]

# 3) Split each line into tokens by spaces (keeps <s> and </s>)
# Use only first 3 lines as training corpus (excludes the test sentence at the bottom)
sentences = [line.split() for line in lines[:3]]

print(sentences)
```

✓ 0.0s

```
[['<s>', 'He', 'read', 'a', 'book', '</s>'], ['<s>', 'I', 'read', 'a', 'different', 'book', '</s>'], ['<s>', 'He', 'read', 'a', 'book', 'by', 'Danielle', '</s>']]
```

Figure 24: Text Corpus Pre-Processing

This cell preprocesses the raw file content by splitting the text into individual lines and filtering out non-sentence lines such as headers and separators. The process keeps all lines that start with the sentence boundary token. The process tokenises each kept line by using whitespace as the splitting point while treating and boundary markers as separate tokens. The training corpus

consists of three lines from the text which prevents model training on the test sentence that appears at the end of the document.

Data Transformation

```
from collections import Counter

# Flatten all tokens into one list
all_tokens = [tok for sent in sentences for tok in sent]

# Count unigrams (single tokens)
unigram_counts = Counter(all_tokens)

# Count bigrams (pairs)
bigram_counts = Counter()
for sent in sentences:
    for i in range(len(sent) - 1):
        bigram_counts[(sent[i], sent[i+1])] += 1

print("count(<s>) =", unigram_counts["<s>"])
print("count(<s>, I) =", bigram_counts[("<s>", "I")])
```

✓ 0.0s

```
count(<s>) = 3
count(<s>, I) = 1
```

Figure 25: Data Transformation

The cell transforms all training sentences into one complete token list while it creates two frequency dictionaries which include `unigram_counts` that counts the total appearances of each single token and `bigram_counts` that counts the total appearances of every adjacent word pair. The two frequency dictionaries which the program generates are essential for computing bigram probabilities.

Sentence Probability Function

```
def sentence_probability(tokens, unigram_counts, bigram_counts, V, smoothing=False):
    prob = 1.0

    for i in range(1, len(tokens)):
        prev_word = tokens[i-1]
        word = tokens[i]

        bigram = (prev_word, word)

        if smoothing:
            numerator = bigram_counts[bigram] + 1
            denominator = unigram_counts[prev_word] + V
        else:
            numerator = bigram_counts[bigram]
            denominator = unigram_counts[prev_word]

        # If a bigram was never seen, probability becomes 0
        if numerator == 0:
            return 0.0

        prob *= numerator / denominator

    return prob
```

✓ 0.0s

Figure 26: Sentence Probability

This cell defines a function that computes the probability of a sentence under the bigram language model by iterating through each consecutive word pair and multiplying their conditional probabilities together, using the standard MLE formula $C(w_{(i-1)}, w_i) / C(w_{(i-1)})$ for the unsmoothed model, or the Laplace formula $(C(w_{(i-1)}, w_i) + 1) / (C(w_{(i-1)}) + V)$ for the smoothed model, and immediately returning 0.0 if any unseen bigram is encountered in the unsmoothed case.

Results

```
test_sentence = "<s> I read a different book by Danielle </s>".split()

V = len(set(all_tokens)) # vocabulary size

unsmoothed = sentence_probability(test_sentence, unigram_counts, bigram_counts, V, smoothing=False)
smoothed = sentence_probability(test_sentence, unigram_counts, bigram_counts, V, smoothing=True)

print("Vocabulary size V =", V)
print("Unsmoothed probability =", unsmoothed)
print("Smoothed probability   =", smoothed)
```

✓ 0.0s

Vocabulary size V = 10
 Unsmoothed probability = 0.037037037037037035
 Smoothed probability = 1.2244070205767176e-06

Figure 27: Results

This cell tokenises the test sentence, computes the vocabulary size $V = 10$, then calls the `sentence_probability` function twice — once without smoothing and once with Laplace smoothing — producing results that exactly match the manual calculations: the unsmoothed

probability of $1/27 \approx 0.03704$ (from $1/3 \times 1 \times 1 \times 1/3 \times 1 \times 1/3 \times 1 \times 1$) and the smoothed probability of $512/418,161,601 \approx 1.224 \times 10^{-6}$ (from $2/13 \times 2/11 \times 4/13 \times 2/13 \times 2/11 \times 2/13 \times 2/11 \times 2/11$), confirming the implementation is correct.

2.4.4 Results and Analysis

Summary

Model	Sentence Probability
Unsmoothed Bigram	$1/27 \approx 0.03704$
Smoothed Bigram (Laplace +1)	$512/418,161,601 \approx 1.224 \times 10^{-6}$

The unsmoothed model produces a higher probability for this test sentence because all bigrams in it were seen during training. However, any single unseen bigram would reduce the probability to zero. Laplace smoothing eliminates this zero-probability problem by assuming every possible bigram has been seen at least once, producing a non-zero probability for any test sentence.

2.5 (Q5) Alternative Approach Implementation

2.5.1 Alternative Tokenization Method: Scikit-Learn (Muhammad Irfan bin Mohd Rizal – TP078491)

2.5.1.1 Implement tokenization using an alternative approach

As an alternative to the group's tokenization approach, Scikit-learn's CountVectorizer was used to perform token extraction.

```
from sklearn.feature_extraction.text import CountVectorizer

with open('../Part_A/data/Data_1.txt', 'r', encoding='utf-8') as file:
    text_data = file.read()

# 1. Initialize and Fit
vect = CountVectorizer()
corpus = [text_data]
X = vect.fit_transform(corpus)

# 2. Extract items from the vocabulary
# .items() gives us both the 'word' and the 'alphabetical_index'
# It follows the discovery order (how they first appear in the text)
vocab_items = list(vect.vocabulary_.items())

# 3. Table Output Display
print(f"{'No.':<5} | {'Token':<20} | {'Assigned ID (Alphabetical)'}")
print("-" * 55)

# Show the first 20 tokens found in the text
for i, (token, alpha_idx) in enumerate(vocab_items[:20]):
    # i+1 is no of token
    # token is word
    # alpha_idx is assigned number to token alphabetically
    print(f" {i+1:<4} | {token:<20} | {alpha_idx}")
```

Figure 28: Initialization and CountVectorizer Tokenization Code

CountVectorizer tokenizes the text automatically while also preparing it for machine learning applications by converting the corpus into a document-term matrix. In this process, the text is first normalized into lowercase, punctuation is removed by default, and valid words are extracted as tokens. The output also includes an assigned ID for each token. This assigned ID represents the position of the token in Scikit-learn's internal vocabulary dictionary, which is arranged alphabetically rather than by order of appearance in the text. Therefore, the assigned ID is used as a feature index for numerical representation in the matrix, not as a sequential token number from the original sentence.

No.	Token	Assigned ID (Alphabetical)
1	classification	9
2	is	25
3	the	43
4	task	41
5	of	36
6	choosing	7
7	correct	12
8	class	8
9	label	28
10	for	16
11	given	18
12	input	21
13	in	20
14	basic	5
15	tasks	42
16	each	14
17	considered	11
18	isolation	26
19	from	17
20	all	1

Figure 29: CountVectorizer Tokenization Output

2.5.1.2 Compare and Contrast between the Alternative Approach and the Group's Approach

The group's approach using NLTK focuses on linguistic tokenization, where words and punctuation are separated based on language rules and the original order of tokens is preserved. This makes NLTK more suitable for analysing sentence structure, punctuation usage, and other language-related characteristics. In contrast, Scikit-learn's CountVectorizer is designed primarily for feature extraction in machine learning. It automatically converts all text to lowercase, removes punctuation, and stores only unique tokens in its vocabulary. Unlike NLTK, its output is not intended to preserve the original sequence of words, but rather to map tokens into numeric feature indices for further modelling. Thus, while both approaches perform tokenization, they differ significantly in purpose and output structure.

2.5.1.3 Explanation of Why the Alternative Approach is Better, Worse, or Different

The Scikit-learn approach is best described as different rather than strictly better or worse. Its main strength lies in its suitability for text classification and machine learning tasks, as it directly transforms text into a structured numerical format that can be used by predictive models. This makes it efficient and practical, especially for large datasets. However, compared to NLTK, it is less suitable for detailed linguistic analysis because it removes punctuation, converts all words to lowercase, and does not preserve token order in the same way. In addition, the assigned IDs generated by CountVectorizer are simply vocabulary indices and do not carry linguistic meaning by themselves. Therefore, if the objective is grammatical or contextual text analysis, NLTK is more appropriate; if the objective is machine learning-based text representation, Scikit-learn is more suitable.

2.5.2 Alternative Tokenization Method: spaCy (Ramaneiss – TP070818)

2.5.2.1 Implement tokenization using an alternative approach.

Another alternative tokenization approach that had been implemented is the spaCy to conduct the token acquisition.

```
# Import spaCy library
import spacy

# Read & Display Data_1.txt
with open("../Part_A/data/Data_1.txt", "r", encoding="utf-8") as f:
    paragraph = f.read().strip()

print("Data_1.txt:")
print(paragraph, "\n")
```

✓ 15.0s

Figure 30: Importing spaCy libraries and Read Dataset

```
Data_1.txt:
Classification is the task of choosing the correct class label for a given input. In basic
classification tasks, each input is considered in isolation from all other inputs, and the set of labels is defined in advance. The basic classification ta
```

Figure 31: Dataset Confirmation Output

```

● # Q5.1: Implement tokenization using an alternative approach (spaCy)
print("Q5.1: Implement tokenization using an alternative approach (spaCy)")

# English-language model for spaCy
nlp = spacy.load("en_core_web_sm")

# Tokenize the paragraph
doc = nlp(paragraph)

# Display spaCy tokens
spacy_tokens = [token.text for token in doc]
print(f"\nTotal tokens (spaCy): {len(spacy_tokens)}")
print("\n Acquired spaCy tokens :")
print(spacy_tokens)
print("\n")

```

Figure 32: spaCy Tokenization Code

```

Q5.1: Implement tokenization using an alternative approach (spaCy)

Total tokens (spaCy): 97

Acquired spaCy tokens :
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'In', 'basic', '\n', 'clas

```

Figure 33: spaCy Tokenization Output

Output:

Q5.1: Implement tokenization using an alternative approach (spaCy)

Total tokens (spaCy): 97

Acquired spaCy tokens :

```

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given',
'input', '.', 'In', 'basic', '\n', 'classification', 'tasks', ',', 'each', 'input', 'is', 'considered', 'in', 'isolation',
'from', 'all', 'other', 'inputs', ',', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', '.', 'The',
'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', '.', 'For', 'example',
',', 'in', 'multiclass', 'classification', ',', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels',
',', 'in', 'open', '-', 'class', 'classification', ',', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in',
'advance', ',', 'and', 'in', 'sequence', 'classification', ',', 'a', 'list', 'of', 'inputs', 'are', 'jointly',
'classified', '.']

```

2.5.2.2 Compare and Contrast between the Alternative Approach and the Group's Approach

The similarity is that both NLTK's `word_tokenize` and spaCy's `en_core_web_sm` correctly tokenize the text in `Data_1.txt` into individual linguistic units into meaningful content words like "Classification", "isolation", and "multiclass". Plus, both NLTK and spaCy's tokenizer correctly separated punctuation marks ('.' ',' ';') from adjacent words as individual tokens.

Besides that, the first difference between the alternative spaCy tokenization and the group's NLTK tokenization is that hyphenated words were preserved by NLTK like "open-class" as a single token whereas spaCy splits it into three tokens namely 'open', '-', and 'class'. The second difference is whitespace handling where spaCy maintains the newline character '\n' as an explicit token whereas NLTK silently discards it. This has resulted in spaCy producing 97 tokens compared to NLTK's 94 tokens. The third difference is that spaCy operates on a pipeline-based architecture using a pre-trained language model known as 'en_core_web_sm' providing each token with built-in linguistic annotations at once whereas NLTK utilizes a rule-based 'Punkt' tokenizer which requires separate function calls for each additional linguistic task.

2.5.2.3 Explanation of Why the Alternative Approach is Better, Worse, or Different

The alternative approach of utilizing spaCy tokenization is just different. This is because spaCy is considerably more convenient and consistent than NLTK for projects that require multiple NLP sub-tasks beyond tokenisation as spaCy can load a complete trained language model through a single `nlp(paragraph)` call to perform tokenisation, Part-of-Speech tagging, dependency parsing and Named Entity Recognition simultaneously compared to NLTK where each component from tokenizer to POS tagger must be individually imported and integrated together. Another reason is that spaCy is naturally more robust and predictable when handling edge cases like URLs, email addresses and special characters since the tokenizer is built around defined English linguistic rules which includes prefix, suffix, and infix rules compared to NLTK's Punkt tokenizer which is trained statistically on general corpora and handles common English well but produce less consistent results on domain-specific or unconventional inputs. Besides that, spaCy's retention of the '\n' newline token reflects its lossless tokenisation design useful in applications such as document reconstruction or code analysis however carries no linguistic or semantic value for sentiment analysis or text classification in this assignment.

2.5.3 Alternative Tokenization Method: HuggingFace (Voon Kai Wen – TP077157)

2.5.3.1 Implement tokenization using an alternative approach

The third alternative tokenization approach executed is the HuggingFace WordPiece tokenization to successfully perform the token extraction.

```
%pip install transformers
```

Figure 34: HuggingFace Transformer Installation

```
Collecting transformers
  Downloading transformers-5.1.0-py3-none-any.whl.metadata (31 kB)
Collecting huggingface-hub<2.0,>=1.3.0 (from transformers)
  Downloading huggingface_hub-1.4.1-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: numpy>=1.17 in c:\users\user\anaconda3\envs\nltk2\lib\site-packages (from transformers) (2.3.5)
Requirement already satisfied: packaging>=20.0 in c:\users\user\anaconda3\envs\nltk2\lib\site-packages (from transformers) (25.0)
Collecting pyyaml>=5.1 (from transformers)
  Downloading pyyaml-6.0.3-cp311-cp311-win_amd64.whl.metadata (2.4 kB)
Requirement already satisfied: regex!=2019.12.17 in c:\users\user\anaconda3\envs\nltk2\lib\site-packages (from transformers) (2025.11.3)
Collecting tokenizers<0.23.0,>=0.22.0 (from transformers)
  Downloading tokenizers-0.22.2-cp39-abi3-win_amd64.whl.metadata (7.4 kB)
Collecting typer-slim (from transformers)
  Downloading typer_slim-0.21.1-py3-none-any.whl.metadata (16 kB)
Collecting safetensors>=0.4.3 (from transformers)
  Downloading safetensors-0.7.0-cp38-abi3-win_amd64.whl.metadata (4.2 kB)
Requirement already satisfied: tqdm>=4.27 in c:\users\user\anaconda3\envs\nltk2\lib\site-packages (from transformers) (4.67.1)
Collecting filelock (from huggingface-hub<2.0,>=1.3.0->transformers)
  Downloading filelock-3.20.3-py3-none-any.whl.metadata (2.1 kB)
Collecting fsspec>=2023.5.0 (from huggingface-hub<2.0,>=1.3.0->transformers)
  Downloading fsspec-2026.2.0-py3-none-any.whl.metadata (10 kB)
Collecting hf-xet<2.0.0,>=1.2.0 (from huggingface-hub<2.0,>=1.3.0->transformers)
  Downloading hf_xet-1.2.0-cp37-abi3-win_amd64.whl.metadata (5.0 kB)
Collecting httpx<1,>=0.23.0 (from huggingface-hub<2.0,>=1.3.0->transformers)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: shellingham in c:\users\user\anaconda3\envs\nltk2\lib\site-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (1.5.4)
...
----- 13/13 [transformers]

Successfully installed anyio-4.12.1 filelock-3.20.3 fsspec-2026.2.0 h11-0.16.0 hf-xet-1.2.0 httpcore-1.0.9 httpx-0.28.1 huggingface-hub-1.4.1 pyyaml-6.0.3
Note: you may need to restart the kernel to use updated packages.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Figure 35: Installation Process and Succession

```

from transformers import AutoTokenizer

# 1. Load the tokenizer
# No need for NLTK or manual string/stopword lists
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

with open('../Part_A/data/Data_1.txt', 'r', encoding='utf-8') as file:
    text_data = file.read()

# 2. Pure Hugging Face Encoding
# 'encode' converts text to Token IDs. 'tokenize' converts text to subword strings.
# Using 'encode_plus' or the __call__ method is the standard for model prep.
encoding = tokenizer(
    text_data,
    add_special_tokens=True,      # Adds [CLS] and [SEP]
    truncation=True,             # Cuts text if it's longer than model max (usually 512)
    padding='max_length',        # Pads to max length (optional, usually for batching)
    return_attention_mask=True   # Tells the model which tokens are real vs padding
)

# 3. Accessing the results
hf_tokens = tokenizer.tokenize(text_data) # The subword strings
hf_ids = encoding['input_ids']           # The numerical IDs for the model

print('Hugging Face Tokenization:')
print(f'Total subword tokens: {len(hf_tokens)}')
print('Sample Tokens:', hf_tokens[:20])
print('Sample IDs:', hf_ids[:20])

# To see how it handles "cleaning" naturally:
print('\nDecoded example (how the model sees it):')
print(tokenizer.decode(hf_ids[:20]))

```

Figure 36: HuggingFace Tokenization Code

```

Hugging Face Tokenization:
Total subword tokens: 97
Sample Tokens: ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'in', 'basic', 'classification', 'tasks', '.']
Sample IDs: [101, 5579, 2003, 1996, 4708, 1997, 10549, 1996, 6149, 2465, 3830, 2005, 1037, 2445, 7953, 1012, 1999, 3937, 5579, 8518]

Decoded example (how the model sees it):
[CLS] classification is the task of choosing the correct class label for a given input. in basic classification tasks

```

Figure 37: HuggingFace Tokenization Output Success

Output:

Hugging Face Tokenization:

Total subword tokens: 97

Sample Tokens: ['classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', '.', 'in', 'basic', 'classification', 'tasks', '.']

Sample IDs: [101, 5579, 2003, 1996, 4708, 1997, 10549, 1996, 6149, 2465, 3830, 2005, 1037, 2445, 7953, 1012, 1999, 3937, 5579, 8518]

Decoded example (how the model sees it):

[CLS] classification is the task of choosing the correct class label for a given input. in basic classification tasks

2.5.3.2 Compare and Contrast between the Alternative Approach and the Group's Approach

The similarity is that both NLTK's `word_tokenize` and HuggingFace's `bert-base-uncased` WordPiece tokenizer were able to tokenize the text in `Data_1.txt` correctly into individual linguistic units that include meaningful words such as "classification", "isolation" and "variants". In addition, both NLTK and HuggingFace's tokenizer were able to split punctuation marks ('.', ',', ';') correctly from words as individual tokens.

Apart from that, the first difference between the alternative HuggingFace tokenization and the group's NLTK tokenization is that HuggingFace's approach is a subword tokenization technique that breaks any word that is not in their vocabulary or is a morphologically complex word into subwords; for example, "multiclass" is broken down into 'multi' and '##class', where '##' is a prefix indicating that this is a continuation of a word that has been broken down into two subwords whereas NLTK's approach treats "multiclass" as a complete word. The second distinction is that HuggingFace's `bert-base-uncased` model converts all the words to lowercase like "Classification" is broken down into 'classification' whereas NLTK's approach treats each word's case as it is written in the original document. The third distinction is that HuggingFace breaks up hyphenated words like "open-class" into three different words namely 'open', '-', and 'class' whereas NLTK treats it as a complete word "open-class". As a result of these three differences, HuggingFace has 97 tokens compared to NLTK's 94 and this is mainly caused by breaking up "open-class" and "multiclass". The fourth distinction is that HuggingFace's approach is a vocabulary-driven WordPiece algorithm that has been pre-trained on a lot of text and can therefore break down any word that is not in its vocabulary by breaking it down into known word pieces whereas NLTK's approach is a rule-based approach that uses a Punkt tokenizer that treats words as atomic units and requires a different function call for different linguistic tasks.

2.5.3.3 Explanation of Why the Alternative Approach is Better, Worse, or Different

The alternative approach using HuggingFace WordPiece tokenization is just different. This is because HuggingFace is far more powerful and flexible than NLTK when it comes to any project that requires any kind of deep learning or transformer-based NLP pipelines because the

tokenizer is designed to directly produce token IDs that can be used as numerical inputs to pre-trained models like BERT, GPT and RoBERTa using a single call to `tokenizer.tokenize(paragraph)` unlike NLTK's `word_tokenize` that produces plain string tokens that need to go through additional encoding steps before they can even begin to be fed into any kind of neural network or language model. Another reason is that HuggingFace is also far more robust when it comes to morphologically complex words, rare words and especially when it comes to words that are completely unseen because the WordPiece algorithm breaks down any word that is completely unseen into known word pieces that are within its pre-trained vocabulary such that no word is ever out-of-vocabulary unlike NLTK's Punkt tokenizer that treats every word as an atomic whole unit that can produce inconsistent results when dealing with domain-specific terminology and newly coined words that are not within its pre-trained corpora. Apart from that, HuggingFace's forced lowercasing via the `bert-base-uncased` model also normalizes the entire token vocabulary such that words like "Classification" and "classification" are treated as the same token which is beneficial for this assignment's text classification and sentiment analysis task because it reduces redundancy however this also results in a permanent and irreversible loss of original casing information that can actually carry semantic meaning in tasks such as named entity recognition where proper nouns must be distinguished from common words based on capitalization.

2.5.4 Alternative Tokenization Method: TextBlob (Sarvein – TP071496)

2.5.4.1 Implement tokenization using an alternative approach

```
# PART 1: Word Tokenization using TextBlob (2 marks)
print("PART 1: WORD TOKENIZATION USING TEXTBLOB")

# Create TextBlob object
blob = TextBlob(sentence)

# Tokenize words
tokens = blob.words

print(f"\nTotal number of tokens: {len(tokens)}\n")
print("Tokenized words:")
print(tokens)
print("\n")
```

Figure 38: TextBlob Initialization and Word Tokenization

```

PART 1: WORD TOKENIZATION USING TEXTBLOB
Total number of tokens: 82
Tokenized words:
['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', 'In', 'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', 'For', 'example', 'in', 'multiclass', 'classification', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', 'in', 'open-class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', 'and', 'in', 'sequence', 'classification', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']

```

Figure 39: Word Tokenization Output

PART 1: WORD TOKENIZATION USING TEXTBLOB

Total number of tokens: 82

Tokenized words:

['Classification', 'is', 'the', 'task', 'of', 'choosing', 'the', 'correct', 'class', 'label', 'for', 'a', 'given', 'input', 'In', 'basic', 'classification', 'tasks', 'each', 'input', 'is', 'considered', 'in', 'isolation', 'from', 'all', 'other', 'inputs', 'and', 'the', 'set', 'of', 'labels', 'is', 'defined', 'in', 'advance', 'The', 'basic', 'classification', 'task', 'has', 'a', 'number', 'of', 'interesting', 'variants', 'For', 'example', 'in', 'multiclass', 'classification', 'each', 'instance', 'may', 'be', 'assigned', 'multiple', 'labels', 'in', 'open-class', 'classification', 'the', 'set', 'of', 'labels', 'is', 'not', 'defined', 'in', 'advance', 'and', 'in', 'sequence', 'classification', 'a', 'list', 'of', 'inputs', 'are', 'jointly', 'classified']

2.5.4.2 Compare and Contrast between the Alternative Approach and the Group's Approach

NLTK uses its `word_tokenize()` function to perform text tokenization. In this case, punctuation marks are treated as separate tokens, and hyphenated words are also tokenized separately. Therefore, the overall number of tokens is higher. On the other hand, TextBlob uses its `.words` property to perform text tokenization. In this case, punctuation is excluded from the list of tokens, and hyphenated words such as "open-class" are treated as a single token. Therefore, the output of the text tokenization performed by TextBlob contains only 82 tokens.

2.5.4.3 Explanation of Why the Alternative Approach is Better, Worse, or Different

TextBlob is neither superior to NLTK nor inferior to it in terms of word tokenization; rather, it is simply another tool that can be more suitable depending on the task at hand. For general purposes, including sentiment analysis and text classification, TextBlob is more suitable, as it provides cleaner tokens by eliminating punctuation, thereby resulting in cleaner data. On the other hand, if the task involves structural analysis, including POS tagging, parse tree generation, and syntactic analysis, NLTK would be more suitable, as punctuation acts as a significant sentence boundary marker. Therefore, the appropriateness of using TextBlob as opposed to NLTK depends entirely on the task at hand, and neither of the tools can be said to be superior to the other.

3.0 Part B: Implementation of Text Analytics Techniques and Methodologies

3.1 (Q1) Dataset and Exploratory Data Analysis

3.1.1 Dataset Description

For Part B of this assignment, the dataset selected is the IMDB Dataset of 50,000 Movie Reviews. This dataset was originally created by Maas et al. (2011) and subsequently made publicly available via the Stanford AI Lab (<http://ai.stanford.edu/~amaas/data/sentiment/>) and the Kaggle page (<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>) by Lakshmi N. This is a binary sentiment classification dataset said to contain substantially more data than previous benchmark datasets possessing around 50K highly polar movie reviews allowing an even segregation of 25,000 for training and another 25,000 for testing (Lakshmi N, 2019).

The Kaggle version of the dataset combines two splits into one CSV file which contains two columns namely 'review' which is the raw text of the movie review along with 'sentiment' which is a string label of either positive or negative.

Link to Dataset: [IMDB Dataset.csv](#)

License: [Open Source](#)

3.1.2 Problem Definition

The classification problem that is addressed in this assignment is binary sentiment classification which is the task of automatically determining if an obtained piece of text represents or expresses a positive or negative opinion. To be exact, the problem can be represented with an input movie review text (x) along with a supervised classification model which needs to predict a binary output label of $y \in \{0, 1\}$ where 0 represents a negative sentiment whereas 1 represents a positive sentiment. This is considered to be a document-level polarity sentiment classification problem where the model assigns a single sentiment label to the entire review rather than to individual sentences or aspects within it.

Plus, sentiment classification is a well-established subfield within the broader Natural Language Processing (NLP) and opinion mining domains both with wide practical applications. Within the entertainment or film domain, understanding the audience sentiment towards films and movies is highly valuable to studios, producers, film distributors and also online streaming platforms to assist in effectively gauging audience reaction, guide in making more informed and data-backed marketing decisions along with improving content recommendation systems for a more personalized and tailored approach.

3.1.3 Literature Review

Domain 1: Early Foundations: Lexicon-Based and Classical Machine Learning Approaches

The domain of sentiment analysis on text documents was pioneered by Pang, Lee, and Vaithyanathan (2002) who worked on the problem of classifying text documents not by topic but by overall sentiment in determining whether the review is positive or negative. They used movie reviews as their data and found out that machine learning techniques definitely outperform human-produced baselines. Referencing their ‘Thumbs Up? Sentiment Classification Using Machine Learning Techniques’ paper, the authors used three machine learning methods namely Naïve Bayes, Maximum Entropy, and SVM (Support Vector Machines) with bag-of-words features on a corpus of 1,400 movie reviews. It was deduced that sentiment classification remained harder than topic-based categorisation despite outperforming human baselines (Pang et al., 2002).

Branching from this foundation, the introduction of the IMDB Large dataset by Maas et al. (2011) marked a major milestone in the development of sophisticated datasets. This is because the resulting dataset comprised 50,000 reviews divided into 25,000 training samples and 25,000 test samples became the de facto standard for movie review sentiment analysis. In addition, Maas et al. also proposed the learning word vectors capturing semantic and sentiment information achieving 88.89% classification accuracy. This study had shifted research towards distributed word representations along with opening the door for neural methods (Maas et al., 2011).

Domain 2: Word Embeddings and Distributed Text Representations

The shift from sparse high-dimensional feature vectors towards more dense and continuous word representations has been one of the most significant milestones in NLP history. Specifically, the Word2Vec model first proposed by Tomas Mikolov and his team at Google Brain in 2013 has been a major breakthrough in NLP (IEEE Signal Processing Society, 2021). The model leverages the power of Continuous Bag of Words (CBOW) and Continuous Skip-gram architectures to efficiently learn word embeddings from large corpora thus becoming widely adopted for its simplicity and robustness (Mikolov et al., 2013). For sentiment analysis, Word2Vec embeddings are also very useful as words with similar semantic relationships like ‘brilliant’ and ‘outstanding’ are placed together in the embedding space so that classifiers can generalise across synonymous sentiment expressions without requiring explicit lexicon engineering.

It is known that two of the most successful deep learning methods for word embedding in sentiment analysis are Word2Vec proposed by Tomas Mikolov and Global Vectors (GloVe) proposed by Pennington, Socher, and Manning in 2014. GloVe is said to leverage both local and global word co-occurrence statistics to construct word embedding with outstanding results capturing subtle semantic relationships or similarities (Jain, 2024). Unlike Word2Vec trained using a local context, GloVe explicitly models corpus-wide co-occurrence patterns suited for capturing global statistics of word co-occurrences across large text corpus like the IMDB dataset review text using matrix factorization. Both embedding methods have since been widely adopted as input representations for CNN and LSTM-based text classifier models.

Domain 3: Deep Learning Approaches and Advances

The mid-2010s witnessed a shift from hand-crafted feature engineering to end-to-end deep learning architectures bringing significant improvements towards sentiment analysis. For example, Kim (2014) showcased that utilizing Convolutional Neural Networks (CNNs) on pre-trained word vectors could achieve strong sentence-level classification performance even with little hyperparameter tuning and static vectors (Kim, 2014). Plus, another study by Dai & Le in 2015 utilized Long Short-Term Memory (LSTM) pretrained by recurrent language models and sequence autoencoders to be more effective than previous baselines in capturing sequential data and dependencies in reviews such as acquiring a test error rate of 7.24% on the IMDB movie sentiment dataset (Dai & Le, 2015).

Recently, more studies have also ventured into hybrid architectures like combining CNNs and LSTMs together to leverage both local feature extraction and sequential modelling capabilities. A study by Islam et al. (2024) had conducted a comparative analysis of CNN, LSTM, CNN-LSTM and BERT models that are coupled with Deep Neural Networks (DNNs) on the IMDB 50K dataset which recorded that the hybrid CNN-LSTM model performed better than either CNN or LSTM standalone architectures while the pre-trained transformer models consistently outperformed the other models (Islam et al., 2024).

Domain 4: Hyperparameter Tuning and Model Optimisation

Hyperparameter tuning is an important aspect in achieving optimal performance in sentiment classification models. For conventional machine learning models like Logistic Regression and SVM, GridSearchCV and RandomizedSearchCV from scikit-learn are the standard approaches of exploring hyperparameter spaces. For example, a study from 2022 had implemented hyperparameter optimisation in SVM-based sentiment analysis on the IMDB dataset showed some improvement in performance after tuning the regularization strength and kernel parameters compared to the default parameters values (Valdata, 2022). Another study by Faridi et al. was conducted by applying hyperparameter tuning to Logistic Regression algorithm using Grid Search and Active Learning which had reported an increased accuracy of 90.9% along with 90.58% for Active Learning alone and 90.17% for Grid Search alone which are still higher than the default Logistic Regression model with 89.81% (Faridi et al., 2025).

Additionally, hyperparameters tuning for deep learning models cover a wide range of parameters like the learning rate, batch size, number of epochs, dropout rate along with architecture-specific parameters like the number of hidden layers and units. At the same time, new hyperparameter tuning methods have been discovered and implemented in recent studies such as the paper by Valera & Mehta (2025) which adopted Bayesian optimisation and tools like Optuna for optimizing hyperparameters more proficiently and reducing computational costs proven while also achieving a superior accuracy of 99.87% to reveals outstanding sentiment classification abilities at macro and micro levels (Valera & Mehta, 2025).

3.1.4 Justification of Dataset Selection

The selection of the IMDB 50,000 Movie Reviews dataset as the chosen dataset for this assignment can be highly justified with the following reasons:

1. Alignment with the Assignment Objective

This is because the assignment specifically requires to build an effective Supervised Text Classification model. Hence, the chosen IMDB dataset directly supports this objective and purpose by providing the necessary text data namely the movie reviews along with clear binary class labels between positive and negative possessing the necessary features for building these models. Hence, this reduces the need for extensive label engineering, additional annotation or transformation of the target variable.

2. Academic Credibility and Citability

The original dataset is recorded to originate from a peer-reviewed ACL 2011 publication by Maas et al. from Stanford University which makes the dataset an academically rigorous but also reputable and legitimate data source. This ensures that the data is not an arbitrarily collected web scrape but a deliberately curated corpus designed specifically as a benchmark for NLP research to effectively validate the scientific rigour of any experiment conducted using it.

3. Suitability for Transformer Fine-Tuning

The chosen IMDB dataset containing 50,000 labelled examples undeniably provides more than sufficient volume of training data to meaningfully build and train the text classification models without the risk of overfitting. Plus, the dataset will effectively enable robust evaluation of various deep learning approaches especially since it became the de facto standard for movie review sentiment analysis and also enabled more robust evaluation of deep learning approaches as they emerged throughout the 2010s (Gosai et al., 2026).

4. Benchmarking and Comparability

It is also observed that many published research and studies had utilized the chosen IMDB dataset following the previous statement where it is considered as the de facto standard for movie review sentiment analysis. Hence, the results acquired from this

assignment can be directly compared against a rich body of prior work for meaningful and critical analysis of the built models from this assignment. Previous studies will help in fact-checking information, validating acquired findings and identifying potential room for improvements for a more thorough and detailed work.

5. Realistic Text Characteristics

Moreover, the movie reviews within the IMDB dataset are represented as naturally occurring and real-world user-generated text. This is because the text are observed to vary in length, tone, vocabulary, writing style and linguistic complexity which also includes the use of negation, sarcasm, hyperbole and domain-specific film terminology. Such linguistic richness makes the classification task inherently more challenging while also ensuring that the model must learn substantive semantic patterns rather than superficial lexical cues to specific movies to meaningfully test the built model's language understanding capabilities.

3.2 Exploratory Data Analysis (EDA)

3.2.1 Dataset Overview

```
=====
DATASET Overview
=====
Total reviews      : 50,000
Columns             : ['review', 'sentiment', 'label']
Missing values      : 0
Duplicate rows      : 418
=====

Data Types:
review      object
sentiment   object
label       int64
dtype: object

First 3 rows:
```

	review	sentiment	label
0	One of the other reviewers has mentioned that ...	positive	1
1	A wonderful little production. The...	positive	1
2	I thought this was a wonderful way to spend ti...	positive	1

Figure 40: Dataset Overview

The summary for the IMDB review data set from Kaggle reveals that there are 50,000 movie reviews with three columns: review, sentiment, and label, with no missing values, hence

complete for modelling. However, there were 418 duplicate rows, suggesting that there was some duplication in this data set. In terms of data types, both review and sentiment are stored as text (object), whereas label is stored as a 64-bit integer, which is appropriate for classification tasks. The preview of the first three rows confirms that each record contains the review text alongside its sentiment and corresponding numeric label, demonstrating a well-structured dataset ready for preprocessing and supervised sentiment classification using the IMDb review data.

3.2.2 Vocabulary Analysis

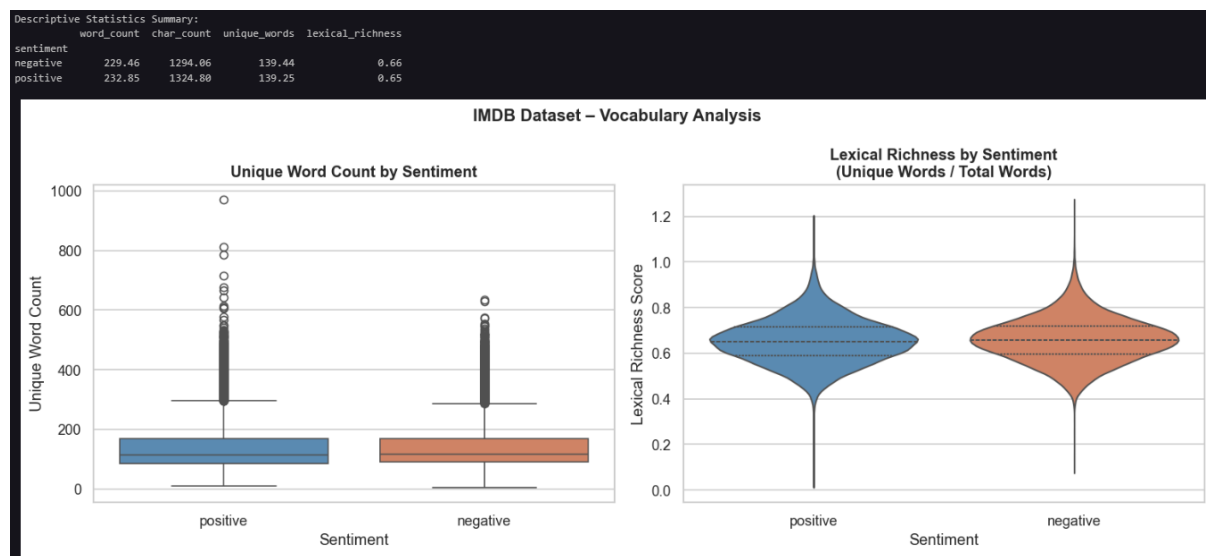


Figure 41: Visualization Charts

From the vocabulary analysis of the data in the IMDb dataset, we can see that positive and negative reviews are almost similar in terms of word count. On average, positive reviews are composed of 232.85 words, while negative reviews are composed of 229.46 words. Moreover, positive reviews have a lexical richness score of 0.65, while negative reviews have a lexical richness score of 0.66. The box plot shows that the number of unique words in positive and negative reviews is almost similar in terms of their medians, which are approximately between 125 to 150. There are a few positive reviews that have extreme outliers, using almost 950 words.

Moreover, we can see from the violin plot that the lexical richness is almost similar for positive and negative reviews. It is clear from the plot that most reviews are almost similar in terms of lexical richness, which is between 0.4 and 0.8. Since positive and negative reviews are almost

similar in terms of vocabulary richness and unique words, we cannot use vocabulary to differentiate between positive and negative reviews.

3.2.3 Overall Summary

```

=====
EDA SUMMARY - IMDB DATASET
=====
Total reviews      : 50,000
Positive reviews   : 25,000 (50%)
Negative reviews   : 25,000 (50%)
Missing values     : 0
Duplicate rows     : 418

--- Review Length (Word Count) ---
Overall mean       : 231.2 words
Overall median     : 173.0 words
Min / Max          : 4 / 2470

--- Per Sentiment (Mean Word Count) ---
Positive           : 232.8 words
Negative           : 229.5 words

--- Vocabulary ---
Avg unique words/review: 139.3
Avg lexical richness  : 0.657

```

Figure 42: EDA Summary Table

The EDA for IMDB data indicates that there are 50,000 total reviews, with 25,000 positive and 25,000 negative reviews. This indicates that the data is balanced and does not have any class imbalance. There are no missing values in the data, but there are 418 duplicate rows. These should be removed during preprocessing to avoid any bias in the data. On average, each review contains 231.2 words, but the median is only 173.0 words. This indicates that the distribution is right skewed because of some long reviews. The range is also large, from 4 to 2,470 words. On average, positive reviews contain 232.8 words, while negative reviews contain 229.5 words. On average, each review contains 139.3 unique words, and the lexical richness is moderate at 0.657.

3.2.6 Chosen Predictive Models

Four good options for predicting sentiment were found from the initial analysis of the IMDB dataset. The first is the **Support Vector Machine (SVM)**, which is a popular method that works well for large text feature spaces. The second is the **Long Short-Term Memory (LSTM)**, which is a recurrent neural network that is able to comprehend the order of the text. The third is the **Random Forest Model** which is a very effective method for text data that is also able to comprehend the meaning. The fourth one is the **Logistic Regression** model, which

is, A good option for text classification as it is a well-known supervised algorithm that performs well with high-dimensional TF-IDF features and also gives a direct probability estimate for each sentiment class.

3.3 Pre-processing

3.3.1 Remove Duplicate Data

```
# — Remove duplicate reviews —————
before = len(df)
df = df.drop_duplicates(subset=['review'])
after = len(df)

print(f"Rows before      : {before:,}")
print(f"Rows after       : {after:,}")
print(f"Duplicates removed : {before - after:,}")

# — Overwrite original dataset with cleaned version —————
df.to_csv('IMDB Dataset.csv', index=False)
print(f"Saved {len(df):,} rows to 'IMDB Dataset.csv'")
✓ 1.1s

Rows before      : 50,000
Rows after       : 49,582
Duplicates removed : 418
Saved 49,582 rows to 'IMDB Dataset.csv'
```

Figure 43: Duplicate Data Removal Code

A total of 418 duplicate reviews were identified during the preprocessing stage. Once these were removed, the data set reduced from 50,000 to 49,582, and this updated data set was written back to the original data set for further modelling steps.

3.3.2 Final Dataset Summary

```
# — Pre-Processed Dataset —————
print("Missing values:")
print(df.isnull().sum())
print()
print(f"Remaining duplicates : {df.duplicated(subset=['review']).sum()}")
print(f"Total rows remaining : {len(df):,}")
✓ 0.0s

Missing values:
review      0
sentiment   0
label       0
dtype: int64

Remaining duplicates : 0
Total rows remaining : 49,582
```

Figure 44: Final Dataset Summary

After duplicates were removed, it was verified that the data set was clean, with no missing data in any of the three columns, no duplicates, and 49,582 reviews ready for modelling.

3.4 (Q2) Supervised Text Classification

3.4.1 Support Vector Machine using LinearSVC

3.4.1.1 Model

Library Imports

```
# --- Imports ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings('ignore')

# Text preprocessing
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Feature extraction
from sklearn.feature_extraction.text import TfidfVectorizer

# Model
from sklearn.svm import LinearSVC

# Evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
)

# Download required NLTK data
nltk.download('stopwords', quiet=True)
```

Figure 45: Initializing Libraries

In the above code snippet, all the necessary libraries are being imported before the start of the model. Libraries like pandas, numpy, matplotlib, and seaborn are being imported for the data. The re library is being imported for text cleaning purposes. The warnings library is also being imported. For text cleaning purposes, the necessary libraries like nltk and stopwords are being imported. Porter Stemmer is also being imported. TfidfVectorizer is being imported from the scikit-learn library for converting the text into features. LinearSVC is being imported from the scikit-learn library for the SVM classifier. The necessary libraries like train_test_split and various functions from the evaluation library are being imported for the evaluation of the trained model. The last line of the code is for downloading the stopwords list from the internet. It is being imported for text cleaning purposes.

Load Dataset

```
# -- Load Dataset --

df = pd.read_csv('IMDB Dataset.csv')

# Encode label: positive = 1, negative = 0
df['label'] = df['sentiment'].map({'positive': 1, 'negative': 0})

print(f"Dataset shape : {df.shape}")
print(f"Class distribution:")
print(df['sentiment'].value_counts())
df.head(3)
```

Dataset shape : (49582, 3)

Class distribution:

sentiment	count
positive	24884
negative	24698

Name: count, dtype: int64

	review	sentiment	label
0	One of the other reviewers has mentioned that...	positive	1
1	A wonderful little production. The...	positive	1
2	I thought this was a wonderful way to spend ti...	positive	1

Figure 46: Load Dataset

In this cell, the IMDB dataset is imported and loaded into the notebook using pandas and then encoded into numerical labels, where positive is represented by 1 and negative by 0, ready for use by the SVM classifier. The output shows that the dataset contains 49,582 reviews in three columns, of which 24,884 are positive and 24,698 are negative, suggesting a near-balanced class distribution.

Text Pre-Processing

```
# -- Text Preprocessing --
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def clean_text(text):
    # 1. Remove HTML tags
    text = re.sub('<.*>', '', text)
    # 2. Remove non-alphabetic characters
    text = re.sub('[^a-zA-Z]', '', text)
    # 3. Lowercase and tokenize
    tokens = text.lower().split()
    # 4 & 5. Remove stop words and stem
    tokens = [stemmer.stem(w) for w in tokens if w not in stop_words]
    return ' '.join(tokens)

df['clean_review'] = df['review'].apply(clean_text)
print(f"Done. Sample cleaned review:\n\n{df['clean_review'].iloc[0][:300]}")
```

✓ 55s

Done. Sample cleaned review:

one review mention watch oz episod hook right exactli happen first thing struck oz brutal unflinch scene violenc set right word go trust show faint heart timid show pull pur

Figure 47: Text Pre-Processing & Cleaning Code

In this cell, a text cleaning function is defined. This function is designed to clean the text of each review before it is input into the model. The function removes HTML tags, non-alphabetic characters, makes all text lowercase, removes stopwords, and uses Porter Stemming to reduce words to their base form. This function is then applied to all reviews in the dataset, and the cleaned text is stored in a new column named `clean_review`.

Test & Train Split

```
# — Train/Test Split & TF-IDF Vectorisation —
X = df['clean_review']
y = df['label']

# 80/20 stratified split to preserve class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# TF-IDF: convert text to numerical feature matrix
# max_features=40000 adjusted for cleaned dataset size
# ngram_range=(1,2) includes unigrams and bigrams
tfidf = TfidfVectorizer(max_features=40000, ngram_range=(1, 2), sublinear_tf=True)

X_train_tfidf = tfidf.fit_transform(X_train) # fit on training data only
X_test_tfidf = tfidf.transform(X_test)      # transform test data

print(f"Training samples : {X_train_tfidf.shape[0]}")
print(f"Test samples    : {X_test_tfidf.shape[0]}")
print(f"TF-IDF features   : {X_train_tfidf.shape[1]}")

✓ 19.5s

Training samples : 39665
Test samples     : 9917
TF-IDF features  : 40000
```

Figure 48: Test & Train Split

Using an 80/20 ratio, this cell separates the clean reviews and their labels into training and testing data. This gives us 39,665 training data points and 9,917 test data points. We chose this 80/20 split because we have a lot of data: 49,582 reviews. Even with an 80/20 split, we still have more than 10,000 data points in the testing set, which is more than enough for testing. To make sure that both sets contain the same number of classes as the original dataset, stratified splitting is used. The TF-IDF Vectorizer changes the text data into a collection of 40,000 numerical features, which contains both words and combinations of two words. It is only fitted to the training data to avoid data leaking.

Model Training

```
# — Build & Train SVM Model —

svm_model = LinearSVC(C=1.0, max_iter=1000, random_state=42)

svm_model.fit(X_train_tfidf, y_train)

print("SVM model trained successfully.")

✓ 0.5s

SVM model trained successfully.
```

Figure 49: Baseline SVM Model Training

The LinearSVC classifier is configured in this cell by setting the regularisation parameter to 1.0, denoted by 'C=1.0', the maximum iterations to 1000 to ensure algorithm convergence, and the random state. The SVC classifier is replaced by the LinearSVC classifier due to its superior performance and efficacy on large datasets. The TF-IDF features are already well-separated in the high-dimensional space. It works just as well as more complicated kernels like rbf or polynomial, but without the extra processing effort. Then, the model is trained on the TF-IDF training data, and the output shows that the training went well.

3.4.2 Evaluation Measures

Table 4: Evaluation Measures

Metric	Formula	Description
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	The share of all reviews that the model got right. This figure is reliable because we know that there are an equal number of positive and negative reviews in the IMDB dataset.
Precision	$TP / (TP + FP)$	The share of reviews predicted as positive that are actually positive. A high precision score indicates that the model does not make very many false positives that is, the model does not often predict a negative review to be a positive one.
Recall	$TP / (TP + FN)$	The share of actual positive reviews that the model found. A high recall score indicates that the model does

		not make very many false negatives; that is, the model does not often fail to predict a positive review.
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	The harmonic average of Precision and Recall. This score is the most important one in GridSearchCV because it combines both of the other metrics, and both errors are equally costly to the model.
Confusion Matrix	-	A 2x2 matrix of true positives, true negatives, false positives, and false negatives. This table gives us a good sense of how well the model classifies both positive and negative reviews.

3.4.3 Model Evaluation and Results

```

=====
SVM MODEL EVALUATION RESULTS
=====
Accuracy : 0.9002 (90.02%)
Precision : 0.8977
Recall : 0.9042
F1-Score : 0.9009
=====

Detailed Classification Report:

```

	precision	recall	f1-score	support
Negative	0.90	0.90	0.90	4940
Positive	0.90	0.90	0.90	4977
accuracy			0.90	9917
macro avg	0.90	0.90	0.90	9917
weighted avg	0.90	0.90	0.90	9917

Figure 50: Baseline SVM Model Evaluation Results

In the test dataset, the SVM model accurately classified 9 out of every 10 evaluations, resulting in an overall accuracy of 90.02%. The recall of 0.9042 suggests that the model was able to correctly classify 90.4% of all genuine positive reviews, while the precision of 0.8977 indicates that the model was correct 89.8% of the time when classifying a review as positive. The classification report also indicates that the model was consistent in classifying both types of sentiment since both negative and positive sentiment had a precision, recall, and F1-Score of 0.90.

SVM - Confusion Matrix

True label	Negative	4427	513
	Positive	477	4500
		Negative	Positive
		Predicted label	

Figure 51: Confusion Matrix (Baseline SVM)

The confusion matrix offers a visual representation of the model's predictions for both sentiment classes. The model accurately classified 4,427 of the 4,940 actual negative reviews as negative (true negatives) and incorrectly predicted 513 as positive (false positives). The model correctly classified 4,500 of the 4,977 actual positive reviews as positive (true positives) and incorrectly predicted 477 as negative (false negatives). The model exhibits exceptional performance in both classes, with the majority of predictions lying along the diagonal, which indicates accurate classifications. The SVM model's consistent and balanced performance, as evidenced by the classification report, is further supported by the comparatively low and evenly distributed number of misclassifications between both classes.

3.4.4 (Q3) Hyper parameters

Hyperparameter Selection

GridSearchCV is used to systematically search over a defined parameter grid. A 5-fold cross-validation is applied on the training set for each combination.

Table 5: Hyperparameter Selection (SVM)

Hyperparameter	Values Tested	Description
C (Regularisation Parameter)	0.01, 0.1, 1.0, 10.0	Controls the trade-off between maximising the decision boundary margin and minimising classification errors. A smaller C allows more misclassification in favour of a wider margin, reducing the risk of overfitting. A larger C enforces stricter classification of training samples, which may lead to overfitting on noisy text data.
loss (Loss Function)	hinge, squared_hinge	Defines the penalty for misclassified samples. Squared_hinge is more sensitive to outliers than the hinge loss, which applies a linear penalty. Squared_hinge is the default for LinearSVC and generally produces smoother optimisation convergence.

max_iter (Maximum Iterations)	500, 1000	Specifies the maximum number of iterations allowed for the solver to converge to an optimal solution. A higher value reduces the risk of non-convergence warnings on large, high-dimensional datasets such as the TF-IDF feature matrix derived from 50,000 reviews.
-------------------------------	-----------	--

GridSearchCV Implementation

```
# --- GridSearchCV ---
from sklearn.model_selection import GridSearchCV

# Define the parameter grid to search over
param_grid = {
    'C'      : [0.01, 0.1, 1.0, 10.0],
    'loss'   : ['hinge', 'squared_hinge'],
    'max_iter': [500, 1000]
}

# Initialise the base estimator
svm_base = LinearSVC(random_state=42)

# GridSearchCV with 5-fold cross-validation, scoring on F1 (macro)
# n_jobs=-1 uses all available CPU cores
grid_search = GridSearchCV(
    estimator = svm_base,
    param_grid = param_grid,
    cv        = 5,
    scoring   = 'f1_macro',
    n_jobs    = -1,
    verbose   = 1
)

print("Running GridSearchCV over the following grid:")
print(param_grid)

grid_search.fit(X_train_tfidf, y_train)

print("\nGridSearchCV complete.")
print(f"Best Parameters : {grid_search.best_params_}")
print(f"Best CV F1 Score: {grid_search.best_score_:.4f}")

✓ 213s

Running GridSearchCV over the following grid:
{'C': [0.01, 0.1, 1.0, 10.0], 'loss': ['hinge', 'squared_hinge'], 'max_iter': [500, 1000]}
Fitting 5 folds for each of 16 candidates, totalling 80 fits

GridSearchCV complete.
Best Parameters : {'C': 1.0, 'loss': 'hinge', 'max_iter': 500}
Best CV F1 Score: 0.8999
```

Figure 52: GridSearchCV Hyperparameter Implementation

GridSearchCV is used for searching over a grid of specified parameters. A 5-fold cross-validation is used on the training data for each set of parameter values. This results in 80 fits over 16 different parameter combinations. The parameter grid is defined by three hyperparameters. The parameter C is tested over four values (0.01, 0.1, 1.0, and 10.0) for varying regularization strengths. The penalty function is evaluated by testing the loss parameter over two values: hinge and squared_hinge. The parameter max_iter is tested over two values, namely 500 and 1000, for ensuring convergence. The parameter used for evaluating each

parameter combination is F1-macro, as it balances precision and recall equally for each of the two sentiments. After completion of this process, GridSearchCV identified the parameter combination as $C=1.0$, $\text{loss}=\text{hinge}$, and $\text{max_iter}=500$ for obtaining the best cross-validation F1-score of 0.8999.

Model Evaluation after Tuning

```
# --- Evaluate Best Tuned SVM on Test Set ---
best_svm = grid_search.best_estimator_

y_pred_tuned = best_svm.predict(X_test_tfidf)

acc_t = accuracy_score(y_test, y_pred_tuned)
prec_t = precision_score(y_test, y_pred_tuned)
rec_t = recall_score(y_test, y_pred_tuned)
f1_t = f1_score(y_test, y_pred_tuned)

print("=" * 50)
print("    TUNED SVM MODEL EVALUATION RESULTS")
print("=" * 50)
print(f" Accuracy : {acc_t:.4f} ({acc_t*100:.2f}%)")
print(f" Precision : {prec_t:.4f}")
print(f" Recall   : {rec_t:.4f}")
print(f" F1-Score : {f1_t:.4f}")
print("=" * 50)
print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred_tuned, target_names=['Negative', 'Positive']))
```

✓ 0.0s

```
=====
TUNED SVM MODEL EVALUATION RESULTS
=====
Accuracy : 0.9017 (90.17%)
Precision : 0.8950
Recall    : 0.9110
F1-Score  : 0.9029
=====

Detailed Classification Report:
      precision    recall  f1-score   support

Negative    0.91     0.89     0.90     4940
Positive    0.89     0.91     0.90     4977

accuracy          0.90     0.90     0.90     9917
macro avg         0.90     0.90     0.90     9917
weighted avg      0.90     0.90     0.90     9917
```

Figure 53: Tuned SVM Model Evaluation Results

This best model obtained from GridSearchCV is now used for predicting on the test data set. The accuracy of the tuned SVM model is found to be 90.17%, which is a slight increase from the baseline accuracy of 90.02%. It is evident that the hyperparameter tuning has created a slightly better generalizing model. The precision is slightly reduced from 0.8977 to 0.8950, but recall is increased from 0.9042 to 0.9110, indicating that this tuned model is better in finding real positive reviews with fewer false negatives. The F1-Score is also slightly increased from 0.9009 to 0.9029.

The fine-grained classification report shows that the negative class had a precision of 0.91 and a recall of 0.89. The positive class, on the other hand, had a recall of 0.91 and a precision of 0.89. The updated model's performance is confirmed by the consistent macro and weighted averages across all indicators, and both classes kept an F1-Score of 0.90. This means that the model is neutral and works the same way for both sentiment classes.

Baseline and Tuned Model Comparison

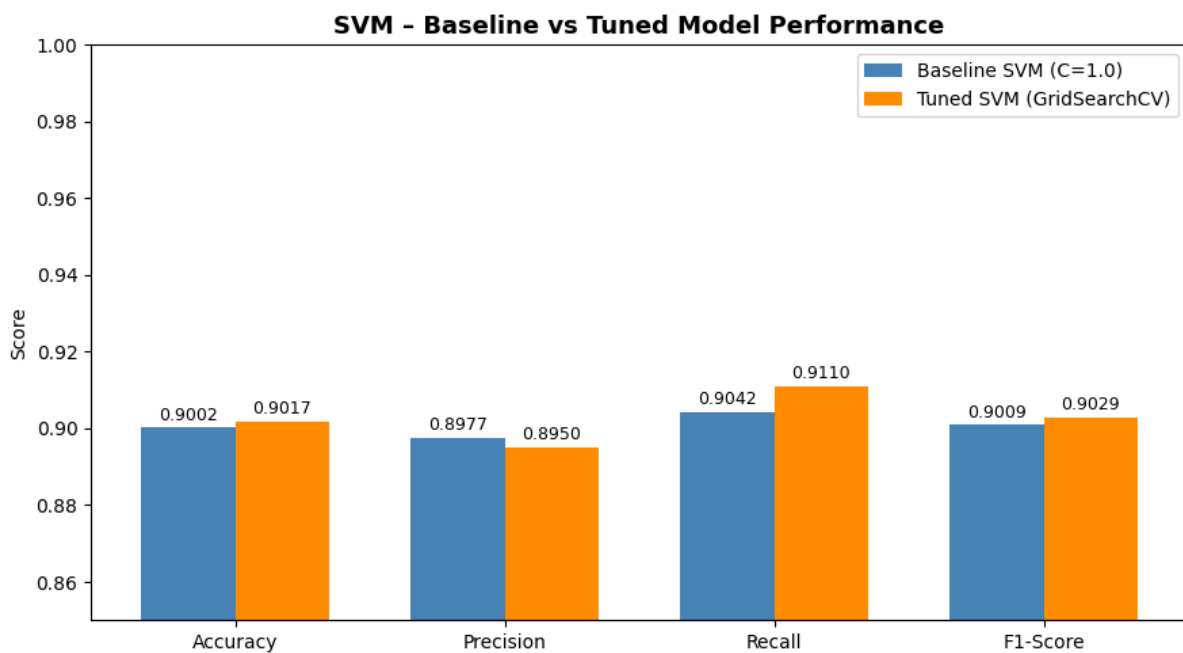


Figure 54: Grouped Bar Chart Visualization of Baseline vs Tuned SVM Model

The bar chart above illustrates a comparison between the baseline SVM model and the tuned SVM model using four different metrics to evaluate the model's performance. In conclusion, it can be noted that the tuned model is slightly better in three out of four metrics. The accuracy improves slightly to 0.9017 from 0.9002. The recall improves to 0.9110 from 0.9042. Similarly, the F1 Score improves to 0.9029 from 0.9009. However, the precision decreases slightly to 0.8950 from 0.8977. This indicates that the tuned model has a slight increase in false positives but also improves true positives. In conclusion, it can be stated that the tuned model has a better set of parameters that enhance the SVM's performance.

3.5 Long Short-Term Memory (LSTM) Model

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) designed to process sequential data such as text. Unlike traditional neural networks, LSTM is capable of capturing long-term dependencies within a sequence by using memory cells and gating mechanisms. This allows the model to retain important contextual information from earlier parts of the text, making it highly suitable for tasks such as sentiment analysis. In this study, LSTM is used to classify text reviews into positive and negative sentiments based on their contextual meaning.

3.5.1 Data Preparation

```
# =====  
# IMPORT REQUIRED LIBRARIES  
# =====  
  
import numpy as np  
import pandas as pd  
# basic data handling  
  
from sklearn.model_selection import train_test_split, RandomizedSearchCV  
# data splitting and hyperparameter tuning  
  
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
# model evaluation  
  
from tensorflow.keras.preprocessing.text import Tokenizer  
from tensorflow.keras.preprocessing.sequence import pad_sequences  
# text processing (convert text to numerical form)  
  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout  
# model building (LSTM neural network)  
  
from tensorflow.keras.callbacks import EarlyStopping  
# training control  
  
from scikeras.wrappers import KerasClassifier  
# connect model with sklearn tuning
```

Figure 55: Importing Libraries

This section imports the required libraries used throughout the model development process. Libraries such as NumPy and pandas are used for basic data handling. Scikit-learn is used for splitting the dataset, evaluating model performance, and performing hyperparameter tuning using RandomizedSearchCV. TensorFlow Keras is used to process the text data through

tokenization and padding, as well as to build and train the LSTM model. EarlyStopping is included to prevent overfitting during training. Lastly, KerasClassifier is used to connect the LSTM model with Scikit-learn so that hyperparameter tuning can be applied.

```
# =====  
# LOAD DATASET  
# =====  
  
df = pd.read_csv("C:/Users/irfan/Downloads/IMDB Dataset.csv")  
# load dataset  
  
df = df[["review", "label"]]  
# select relevant columns (text and target)  
  
print(df.head())  
# preview first few rows  
  
print(df["label"].value_counts())  
# check class distribution
```

Figure 56: Load and Read Dataset

This section loads the dataset used for the analysis. The dataset is read using pandas and stored in a dataframe for further processing. Only the relevant columns, which are the text reviews and their corresponding labels, are selected to focus on the classification task. The first few rows of the dataset are displayed to get a quick understanding of the data, and the distribution of the labels is checked to ensure the dataset is balanced between positive and negative reviews.

```
# =====  
# SPLIT DATA INTO TRAINING AND TESTING SETS  
# =====  
  
X = df["review"] # input (text data)  
y = df["label"]  # target (sentiment label)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,      # 80% training, 20% testing  
    random_state=42,    # ensure consistent results  
    stratify=y          # keep class distribution balanced  
)  
  
print("Train size:", len(X_train))  
print("Test size:", len(X_test))
```

Figure 57: Train and Test Split

The dataset is split into training and testing sets. The text reviews are used as the input features, while the labels represent the target variable for classification. An 80:20 ratio is applied, where 80% of the data is used for training and 20% is used for testing the model. A fixed random state is set to ensure consistent results, and stratification is applied to maintain a balanced distribution of positive and negative labels in both sets.

```
# =====  
# TOKENIZATION AND PADDING  
# =====  
  
max_words = 5000 # limit vocabulary size  
max_len = 100 # set maximum sequence length  
  
tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")  
# create tokenizer to convert text into numbers  
  
tokenizer.fit_on_texts(X_train)  
# learn word index from training data  
  
X_train_seq = tokenizer.texts_to_sequences(X_train)  
X_test_seq = tokenizer.texts_to_sequences(X_test)  
# convert text into sequences of numbers  
  
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding="post")  
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding="post")  
# make all sequences the same length for model input
```

Figure 58: Tokenization and Padding Code

Tokenization is applied to convert the text data into numerical form so that it can be processed by the model, as machine learning models cannot directly understand raw text. A tokenizer is created with a fixed vocabulary size, and it is fitted on the training data to learn the word index. The text reviews are then transformed into sequences of numbers based on this mapping. Since the sequences may have different lengths, padding is applied to ensure that all inputs have the same length. This is necessary because the LSTM model requires a consistent input size, allowing it to process the data efficiently.

3.5.2 Baseline LSTM Model Development

```
# =====  
# FUNCTION TO BUILD LSTM MODEL  
# =====  
from tensorflow.keras import Input  
  
def create_model(embedding_dim=32, lstm_units=32, dropout_rate=0.2):  
    model = Sequential([  
        Input(shape=(max_len,)),  
        Embedding(input_dim=max_words, output_dim=embedding_dim),  
        LSTM(lstm_units),  
        Dropout(dropout_rate),  
        Dense(1, activation="sigmoid")  
    ])  
  
    model.compile(  
        optimizer="adam",  
        loss="binary_crossentropy",  
        metrics=["accuracy"]  
    )  
  
    return model
```

Figure 59: Baseline LSTM Model Development

A function is defined to build the LSTM model with adjustable parameters such as embedding size, number of LSTM units, and dropout rate. The model starts with an embedding layer, which converts the input text into dense vector representations so that the model can better understand relationships between words. This is followed by an LSTM layer that captures sequential patterns and contextual information from the text. A dropout layer is added to reduce overfitting by preventing the model from relying too much on specific features. Finally, a dense output layer with a sigmoid activation function is used to perform binary classification. The model is compiled using the Adam optimizer and binary cross-entropy loss, which are suitable for this type of classification task.

```

# =====
# TRAIN BASELINE LSTM MODEL
# =====

baseline_model = create_model(32, 32)
# build baseline LSTM model with initial parameters

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)
# stop training early if validation performance does not improve

history = baseline_model.fit(
    X_train_pad,
    y_train,
    validation_split=0.2, # use part of training data for validation
    epochs=3,            # number of training cycles
    batch_size=32,       # number of samples per batch
    callbacks=[early_stop],
    verbose=1
)
# train the model

```

Figure 60: Baseline LSTM Model Training

```

Epoch 1/3
992/992 ————— 22s 20ms/step - accuracy: 0.7804 - loss: 0.4582 - val_accuracy: 0.7421 - val_loss: 0.5476
Epoch 2/3
992/992 ————— 19s 19ms/step - accuracy: 0.8611 - loss: 0.3278 - val_accuracy: 0.8468 - val_loss: 0.3620
Epoch 3/3
992/992 ————— 19s 19ms/step - accuracy: 0.8896 - loss: 0.2678 - val_accuracy: 0.8508 - val_loss: 0.3627

```

Figure 61: Baseline LSTM Training Process

The baseline LSTM model is trained using the initial parameter settings defined earlier. Early stopping is applied to monitor the validation loss and stop training when the model performance no longer improves, which helps prevent overfitting and reduces unnecessary training time. A portion of the training data is used as validation data to evaluate the model during training. The model is trained for a small number of epochs with a fixed batch size, allowing it to learn patterns from the data while maintaining a balance between performance and efficiency.

3.5.3 Baseline Model Evaluation

```
# =====
# EVALUATE BASELINE MODEL (TABLE FORMAT)
# =====

from sklearn.metrics import precision_score, recall_score, f1_score
# additional evaluation metrics

y_pred_prob = baseline_model.predict(X_test_pad)
y_pred = (y_pred_prob > 0.5).astype(int)
# convert predicted probabilities into class labels

# --- Metrics ---
baseline_accuracy = accuracy_score(y_test, y_pred)
baseline_precision = precision_score(y_test, y_pred)
baseline_recall = recall_score(y_test, y_pred)
baseline_f1 = f1_score(y_test, y_pred)
# calculate evaluation metrics

# Create summary table
baseline_metrics_df = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Value": [baseline_accuracy, baseline_precision, baseline_recall, baseline_f1]
})

print("=== BASELINE MODEL PERFORMANCE ===")
print(baseline_metrics_df)
# display overall performance

# Confusion Matrix Table
cm = confusion_matrix(y_test, y_pred)

cm_df = pd.DataFrame(
    cm,
    index=["Actual Negative", "Actual Positive"],
    columns=["Predicted Negative", "Predicted Positive"]
)

print("\n=== CONFUSION MATRIX ===")
print(cm_df)
# show prediction breakdown

# Classification report as DataFrame
report = classification_report(
    y_test,
    y_pred,
    target_names=["Negative", "Positive"],
    output_dict=True
)

report_df = pd.DataFrame(report).transpose()

print("\n=== CLASSIFICATION REPORT ===")
print(report_df)
```

Figure 62: Baseline LSTM Model Evaluation

The baseline model is evaluated using several performance metrics to provide a clearer view of how well it classifies positive and negative reviews. Accuracy is used to measure the overall correctness of the model, while precision, recall, and F1-score provide a more detailed evaluation of its classification performance. A confusion matrix is also presented to show the number of correct and incorrect predictions for each class. In addition, the classification report summarizes the model's performance for both negative and positive labels. Using multiple evaluation metrics is important because it gives a more complete understanding of the model rather than relying on accuracy alone.

```

310/310 ————— 2s 6ms/step
=== BASELINE MODEL PERFORMANCE ===
      Metric      Value
0  Accuracy  0.852476
1  Precision  0.828780
2   Recall   0.889894
3  F1-Score  0.858250

=== CONFUSION MATRIX ===
                Predicted Negative Predicted Positive
Actual Negative          4025           915
Actual Positive          548           4429

=== CLASSIFICATION REPORT ===
                precision    recall  f1-score   support
Negative         0.880166    0.814777    0.846210    4940.000000
Positive         0.828780    0.889894    0.858250    4977.000000
accuracy         0.852476    0.852476    0.852476         0.852476
macro avg        0.854473    0.852335    0.852230    9917.000000
weighted avg     0.854377    0.852476    0.852253    9917.000000

```

Figure 63: Baseline LSTM Model Evaluation Results

The baseline LSTM model shows a good overall performance with an accuracy of 0.8525. The recall is slightly higher at 0.8899, indicating that the model is able to correctly identify most of the positive reviews. However, the precision is lower at 0.8288, which suggests that there are some false positive predictions. The F1-score of 0.8583 shows a balanced performance between precision and recall. Based on the confusion matrix, the model correctly classifies a large number of both negative and positive reviews, although there are still some misclassifications in both categories. Overall, the model performs reasonably well as a baseline and provides a good starting point for further improvement through hyperparameter tuning.

3.5.4 (Q3) Hyperparameter Tuning Setup

```
# =====  
# WRAP MODEL FOR RANDOMIZEDSEARCHCV  
# =====  
  
from scikeras.wrappers import KerasClassifier  
# wrapper to connect Keras model with sklearn tools  
  
model = KerasClassifier(  
    model=create_model,  
    verbose=0  
)  
# prepare model for hyperparameter tuning
```

Figure 64: RandomizedSearchCV Hyperparameter Setup

The LSTM model is wrapped using KerasClassifier so that it can be used with Scikit-learn tools such as RandomizedSearchCV. This step is necessary because the original Keras model is not directly compatible with Scikit-learn's tuning methods. By using this wrapper, the model can be treated like a standard machine learning model, allowing different hyperparameter combinations to be tested and evaluated more easily.

```
# =====  
# DEFINE HYPERPARAMETER SEARCH SPACE  
# =====  
  
param_dist = {  
    "model__embedding_dim": [32, 64], # different embedding sizes  
    "model__lstm_units": [32, 64], # different LSTM units  
    "batch_size": [32, 64], # different batch sizes  
    "epochs": [3, 5] # different number of epochs  
}  
# define possible values for tuning
```

Figure 65: Hyperparameter Search Space

Different hyperparameter values are defined to explore how they affect the model's performance. These include the embedding dimension, number of LSTM units, batch size, and number of training epochs. By providing multiple options for each parameter, the tuning process can test different combinations to find a better-performing model. This step is important because the choice of hyperparameters can significantly influence how well the model learns and generalizes to new data.

3.5.5 Hyperparameter Tuning Using RandomizedSearchCV

```
# =====
# PERFORM RANDOMIZED SEARCH CV
# =====

random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=4,          # number of parameter combinations to try
    cv=3,              # number of cross-validation folds
    scoring="accuracy", # evaluation metric
    verbose=2,
    random_state=42
)
# set up hyperparameter tuning using random search

random_search.fit(X_train_pad, y_train)
# train models with different parameter combinations
```

Figure 66: RandomizedSearchCV Hyperparameter Tuning Execution

```
Fitting 3 folds for each of 4 candidates, totalling 12 fits
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=32; total time= 52.0s
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=32; total time= 40.7s
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=32; total time= 37.5s
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=64; total time= 50.6s
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=64; total time= 2.4min
[CV] END batch_size=32, epochs=3, model__embedding_dim=32, model__lstm_units=64; total time= 1.6min
[CV] END batch_size=32, epochs=5, model__embedding_dim=32, model__lstm_units=64; total time= 1.8min
[CV] END batch_size=32, epochs=5, model__embedding_dim=32, model__lstm_units=64; total time= 1.6min
[CV] END batch_size=32, epochs=5, model__embedding_dim=32, model__lstm_units=64; total time= 1.9min
[CV] END batch_size=64, epochs=5, model__embedding_dim=64, model__lstm_units=32; total time= 1.6min
[CV] END batch_size=64, epochs=5, model__embedding_dim=64, model__lstm_units=32; total time= 1.5min
[CV] END batch_size=64, epochs=5, model__embedding_dim=64, model__lstm_units=32; total time= 58.7s
```

Figure 67: Hyperparameter Tuning Process

RandomizedSearchCV is used to perform hyperparameter tuning by testing different combinations of the defined parameters. Instead of trying all possible combinations, it randomly selects a fixed number of combinations, making the process more efficient. Cross-validation is applied to evaluate each combination more reliably by splitting the training data into multiple folds. Accuracy is used as the evaluation metric to determine the best-performing model. This approach helps identify a better set of parameters that can improve the model's performance without requiring excessive computation time.

```
# =====  
# SHOW BEST HYPERPARAMETERS  
# =====  
  
print("Best Parameters:")  
print(random_search.best_params_)  
# display best combination of parameters  
  
print("\nBest Cross-Validation Accuracy:")  
print(random_search.best_score_)  
# display best performance during tuning
```

Figure 68: Best Hyperparameter Tuning Parameters and Accuracy Code

```
Best Parameters:  
{'model__lstm_units': 64, 'model__embedding_dim': 32, 'epochs': 5, 'batch_size': 32}  
  
Best Cross-Validation Accuracy:  
0.8486572718825749
```

Figure 69: Best Hyperparameter Tuning Configuration and Accuracy Output

The best hyperparameter combination is displayed after the tuning process is completed. The results show that the model performs best with 64 LSTM units, an embedding dimension of 32, a batch size of 32, and 5 training epochs. The best cross-validation accuracy achieved is approximately 0.8487, which reflects the model's performance during the tuning process. These results indicate that increasing the number of LSTM units and training epochs helps improve the model's ability to learn patterns from the data.

3.5.6 Tuned Model Evaluation

```
# =====
# EVALUATE TUNED MODEL (TABLE FORMAT)
# =====

best_model = random_search.best_estimator_
# get best model from tuning

from sklearn.metrics import precision_score, recall_score, f1_score
# additional evaluation metrics

y_tuned_pred = best_model.predict(X_test_pad)
# predict using tuned model

# --- Metrics ---
tuned_accuracy = accuracy_score(y_test, y_tuned_pred)
tuned_precision = precision_score(y_test, y_tuned_pred)
tuned_recall = recall_score(y_test, y_tuned_pred)
tuned_f1 = f1_score(y_test, y_tuned_pred)
# calculate evaluation metrics

# Create summary table
tuned_metrics_df = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Value": [tuned_accuracy, tuned_precision, tuned_recall, tuned_f1]
})

print("=== TUNED MODEL PERFORMANCE ===")
print(tuned_metrics_df)
# display overall performance

# Confusion Matrix Table
tuned_cm = confusion_matrix(y_test, y_tuned_pred)
tuned_cm_df = pd.DataFrame(
    tuned_cm,
    index=["Actual Negative", "Actual Positive"],
    columns=["Predicted Negative", "Predicted Positive"]
)

print("\n=== TUNED MODEL CONFUSION MATRIX ===")
print(tuned_cm_df)
# show prediction breakdown

# Classification report as DataFrame
tuned_report = classification_report(
    y_test,
    y_tuned_pred,
    target_names=["Negative", "Positive"],
    output_dict=True
)
tuned_report_df = pd.DataFrame(tuned_report).transpose()

print("\n=== TUNED MODEL CLASSIFICATION REPORT ===")
print(tuned_report_df)
# detailed class-wise performance
```

Figure 70: Tuned LSTM Model Evaluation Results

The tuned LSTM model is evaluated using the same metrics as the baseline model to ensure a fair comparison. The best model obtained from the tuning process is used to make predictions on the test data. Accuracy, precision, recall, and F1-score are calculated to measure overall and

class-wise performance. A confusion matrix is also used to show the number of correct and incorrect predictions, while the classification report provides a more detailed breakdown for each class. Using the same evaluation approach allows a clear comparison to determine whether the tuned model performs better than the baseline model.

```

=== TUNED MODEL PERFORMANCE ===
      Metric      Value
0  Accuracy  0.803368
1  Precision  0.774932
2   Recall   0.857143
3  F1-Score  0.813967

=== TUNED MODEL CONFUSION MATRIX ===
                Predicted Negative  Predicted Positive
Actual Negative                3701                1239
Actual Positive                711                4266

=== TUNED MODEL CLASSIFICATION REPORT ===
                precision    recall  f1-score   support
Negative         0.838849    0.749190    0.791488    4940.000000
Positive         0.774932    0.857143    0.813967    4977.000000
accuracy         0.803368    0.803368    0.803368         0.803368
macro avg        0.806890    0.803167    0.802728    9917.000000
weighted avg     0.806771    0.803368    0.802770    9917.000000

```

Figure 71: Tuned LSTM Model Evaluation Results Output

The tuned LSTM model shows a slightly lower performance compared to the baseline model, with an accuracy of 0.8034. The recall remains relatively high at 0.8571, indicating that the model is still able to identify most of the positive reviews. However, the precision decreases to 0.7749, suggesting an increase in false positive predictions. The F1-score of 0.8140 reflects a balanced but overall lower performance compared to the baseline model. From the confusion matrix, it can be observed that there are more misclassifications, especially in predicting negative reviews. Overall, the tuning process did not improve the model performance and instead resulted in a slight decrease, indicating that the selected hyperparameters may not be optimal for this dataset.

3.5.7 Comparison of Baseline and Tuned Models

```
# =====
# COMPARE BASELINE VS TUNED MODEL
# =====

# create comparison table
comparison_df = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score"],
    "Baseline": [baseline_accuracy, baseline_precision, baseline_recall, baseline_f1],
    "Tuned": [tuned_accuracy, tuned_precision, tuned_recall, tuned_f1]
})

# calculate difference
comparison_df["Difference"] = comparison_df["Tuned"] - comparison_df["Baseline"]

# indicate improvement or decline
comparison_df["Change"] = comparison_df["Difference"].apply(
    lambda x: "Increase" if x > 0 else ("Decrease" if x < 0 else "No Change")
)

# round values for better display
comparison_df["Baseline"] = comparison_df["Baseline"].round(4)
comparison_df["Tuned"] = comparison_df["Tuned"].round(4)
comparison_df["Difference"] = comparison_df["Difference"].round(4)

print("=== BASELINE VS TUNED COMPARISON ===")
print(comparison_df)
```

Figure 72: Baseline vs Tuned LSTM Comparison Code

```
=== BASELINE VS TUNED COMPARISON ===
   Metric  Baseline  Tuned  Difference  Change
0  Accuracy    0.8525  0.8034    -0.0491  Decrease
1  Precision    0.8288  0.7749    -0.0538  Decrease
2   Recall    0.8899  0.8571    -0.0328  Decrease
3  F1-Score    0.8583  0.8140    -0.0443  Decrease
```

Figure 73: Baseline vs Tuned LSTM Comparison Output

The comparison between the baseline and tuned LSTM models shows that the baseline model performs better across all evaluation metrics. The accuracy decreases from 0.8525 to 0.8034, while precision drops from 0.8288 to 0.7749. Similarly, recall decreases from 0.8899 to 0.8571, and the F1-score is reduced from 0.8583 to 0.8140. These results indicate that the tuned model does not improve performance and instead leads to a decline in all metrics. This suggests that the selected hyperparameter combinations are not optimal for this dataset. Overall, the baseline model remains the better-performing model for this task.

3.6 Logistic Regression Model

3.6.1 Data Preparation

Import libraries

```
# -----  
# Import libraries  
# -----  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import time  
  
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix,  
    classification_report,  
    roc_auc_score,  
    roc_curve,  
)  
from sklearn.pipeline import Pipeline  
  
import warnings  
warnings.filterwarnings("ignore")  
  
print("All libraries are imported successfully.")
```

Figure 74: Importing Necessary Libraries

This cell imports all the necessary Python libraries that are needed to build and evaluate the model. For example, pandas and numpy are imported to enable efficient data manipulation and computation whereas matplotlib and seaborn are imported to chart and visualisation outputs. From scikit-learn, the key imports are `train_test_split` for data split, `TfidfVectorizer` for text to number conversion, `LogisticRegression` for the classification model and a all evaluation metrics like `accuracy_score`, `precision_score`, `recall_score` and so on. The `Pipeline` class is also imported to streamline the combination of vectorisation and classification during hyperparameter tuning. Lastly, `warnings.filterwarnings("ignore")` supresses non-critical warning messages to ensure that the output is clean.

Load cleaned Dataset

```

# -----
# Load the IMDB dataset
# -----

df = pd.read_csv("IMDB Dataset.csv")

# Minor dataset inspection
print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
print(f"\nLabel distribution:\n{df['label'].value_counts()}")
print(f"\nSample rows:")
df.head()

```

Figure 75: Load and Read Dataset

```

Dataset shape: (49582, 3)

Column names: ['review', 'sentiment', 'label']

Label distribution:
label
1    24884
0    24698
Name: count, dtype: int64

Sample rows:

```

	review	sentiment	label
0	One of the other reviewers has mentioned that ...	positive	1
1	A wonderful little production. The...	positive	1
2	I thought this was a wonderful way to spend ti...	positive	1
3	Basically there's a family where a little boy ...	negative	0
4	Petter Mattei's "Love in the Time of Money" is...	positive	1

Figure 76: Dataset Load Confirmation and Preview

The IMDB dataset is read using pandas DataFrame via the `pd.read_csv()`. The output indicates that the data has 49,582 rows and 3 columns namely review, sentiment and label. The data is almost a near-balanced distribution with 24,884 reviews belonging to the 'Positive' class and 24,698 reviews belonging to the 'Negative' class making it ideal for binary classification using the Logistic Regression model.

Train (80%) – Test (20%) Split

```
# -----  
# Train / Test Split – 80% training / 20% testing  
# -----  
  
X = df["review"]  
y = df["label"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y,  
)  
  
print(f"Training set size : {len(X_train)}")  
print(f"Testing set size : {len(X_test)}")  
print(f"\nTraining label distribution:\n{y_train.value_counts()}")  
print(f"\nTesting label distribution:\n{y_test.value_counts()}")
```

Figure 77: Train and Test Split

```
Training set size : 39665  
Testing set size : 9917  
  
Training label distribution:  
label  
1    19907  
0    19758  
Name: count, dtype: int64  
  
Testing label distribution:  
label  
1     4977  
0     4940  
Name: count, dtype: int64
```

Figure 78: Train and Test Split Overview

The dataset is split into feature columns ($X = \text{review}$) and target labels ($y = \text{label}$) via a 80/20 split ratio which divides the dataset into 39,665 for training and 9,917 for testing. The `stratify=y` parameter ensures that the class balance of the positive and negative classes is maintained in the training and testing sets thereby avoiding accidental class imbalance in the test or training data.

3.6.2 TF-IDF Vectorization Preparation

TF-IDF Vectorization Configuration

```
# -----
# TF-IDF Vectorization - Convert text to numerical features
# -----

tfidf_vectorizer = TfidfVectorizer(
    max_features=50000,
    ngram_range=(1, 2),
    sublinear_tf=True,
    min_df=2,
    max_df=0.95,
    strip_accents="unicode",
    dtype=np.float32,
)

print("TF-IDF Vectorizer configured:")
print(f" max_features : 50,000")
print(f" ngram_range : (1, 2) - unigrams + bigrams")
print(f" sublinear_tf : True (log scaling)")
print(f" min_df : 2")
print(f" max_df : 0.95")
```

Figure 79: TF-IDF Vectorization Configuration

TF-IDF (Term Frequency-Inverse Document Frequency) converts the raw review text to a numerical sparse matrix where each word or phrase is weighted by its frequency within a review and its overall frequency across all reviews. Words that are frequent within a review but rare are weighted more making them more discriminative for the classification task.

Fitting TF-IDF vectorizer on Training Data

```
# -----
# Fit TF-IDF and Transform the data
# -----

print("Fitting TF-IDF vectorizer on training data")
start_time = time.time()

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

tfidf_time = time.time() - start_time

print(f"\nTF-IDF vectorization complete in {tfidf_time:.2f}s")
print(f" Training matrix shape : {X_train_tfidf.shape}")
print(f" Testing matrix shape : {X_test_tfidf.shape}")
print(f" Vocabulary size : {len(tfidf_vectorizer.vocabulary_)}")
print(f" Matrix density : {X_train_tfidf.nnz / (X_train_tfidf.shape[0] * X_train_tfidf.shape[1]):.4%}")

# Sample feature names (top unigrams and bigrams)
feature_names = tfidf_vectorizer.get_feature_names_out()
print(f"\n Sample unigrams : {[f for f in feature_names[:10]]}")
print(f" Sample bigrams : {[f for f in feature_names if ' ' in f][:10]}")
```

Figure 80: TF-IDF Vectorization Fitting to Training Data

```

Fitting TF-IDF vectorizer on training data

TF-IDF vectorization complete in 32.81s
Training matrix shape : (39665, 50000)
Testing matrix shape  : (9917, 50000)
Vocabulary size       : 50000
Matrix density        : 0.4789%

Sample unigrams : ['00', '000', '000 000', '000 to', '007', '01', '02', '10', '10 000', '10 10']
Sample bigrams  : ['000 000', '000 to', '10 000', '10 10', '10 15', '10 and', '10 because', '10 bethany', '10 br', '10 but']

```

Figure 81: TF-IDF Vectorization Fitting Completion Preview

The vectorizer is fitted only on the training set to prevent data leakage ensuring that the information from the test set does not influence the vocabulary or feature weights. The fitted vectorizer is then used to transform both training and test sets into sparse matrices namely Training matrix shape ($39,665 \times 50,000$), Test matrix shape ($9,917 \times 50,000$), Vocabulary size (50,000 unique terms) and Matrix density (0.4789%).

3.6.3 Baseline Logistic Regression Model Development

Logistic Regression Model Configuration

```

# -----
# Building the Logistic Regression model
# -----

lr_model = LogisticRegression(
    C=1.0,
    solver="lbfgs",
    max_iter=1000,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1,
)

print("Logistic Regression model configured:")
print(f" C (regularisation) : 1.0")
print(f" Solver           : lbfgs")
print(f" Max iterations    : 1000")
print(f" Class weight      : balanced")

```

Figure 82: Logistic Regression Model Configuration & Parameters

The Logistic Regression model is configured with these parameters:

Parameter	Value	Justification
C	1.0	Inverses regularisation strength to manage trade-off between bias and variance
solver	lbfgs	Optimisation algorithm to find the best model weights

max_iter	1,000	Maximum number of iterations allowed for the solver to converge
class_weight	balanced	Automatically adjusts class weights to handle class imbalance
random_state	42	Ensures reproducibility of results
n_jobs	-1	Uses all available CPU cores to speed up training

Baseline Model Training

```
# -----
# Train the model
# -----

print("=" * 60)
print("Training Logistic Regression on IMDB dataset")
print("=" * 60)

start_time = time.time()
lr_model.fit(X_train_tfidf, y_train)
train_time = time.time() - start_time

print(f"\nTraining complete!")
print(f" Training time      : {train_time:.2f}s")
print(f" Number of features: {X_train_tfidf.shape[1]}")
print(f" Number of classes : {len(lr_model.classes_)}")
print(f" Coefficients shape: {lr_model.coef_.shape}")
print(f" Intercept          : {lr_model.intercept_[0]:.4f}")
```

Figure 83: Baseline Logistic Regression Model Training

```
=====
Training Logistic Regression on IMDB dataset
=====

Training complete!
 Training time      : 5.85s
 Number of features: 50000
 Number of classes : 2
 Coefficients shape: (1, 50000)
 Intercept          : 0.1599
```

Figure 84: Baseline Logistic Regression Model Training Confirmation

The model is trained using the function `lr_model.fit()` on the TF-IDF training matrix. As observed, the model has learned 50,000 coefficients (one per feature) stored in a single row coefficient matrix of shape (1, 50,000) with the intercept being 0.1599 highlighting a slight bias towards the Positive class in the absence of any feature.

3.6.4 Baseline Logistic Regression Model Results Evaluation

Most Influential Features Coefficients

```
# -----
# Display most influential features - Top 20 positive and negative indicators
# -----

feature_names = tfidf_vectorizer.get_feature_names_out()
coefficients = lr_model.coef_[0]

top_positive_idx = np.argsort(coefficients)[-20:][::-1]
top_negative_idx = np.argsort(coefficients)[:20]

print("=" * 60)
print("Top 20 Most Influential Features")
print("=" * 60)

print("\n--- Strongest POSITIVE sentiment indicators ---")
for i, idx in enumerate(top_positive_idx, 1):
    print(f" {i:2d}. {feature_names[idx]:<25} (coeff: {coefficients[idx]:+.4f})")

print("\n--- Strongest NEGATIVE sentiment indicators ---")
for i, idx in enumerate(top_negative_idx, 1):
    print(f" {i:2d}. {feature_names[idx]:<25} (coeff: {coefficients[idx]:+.4f})")
```

Figure 85: Most Influential Features Coefficients

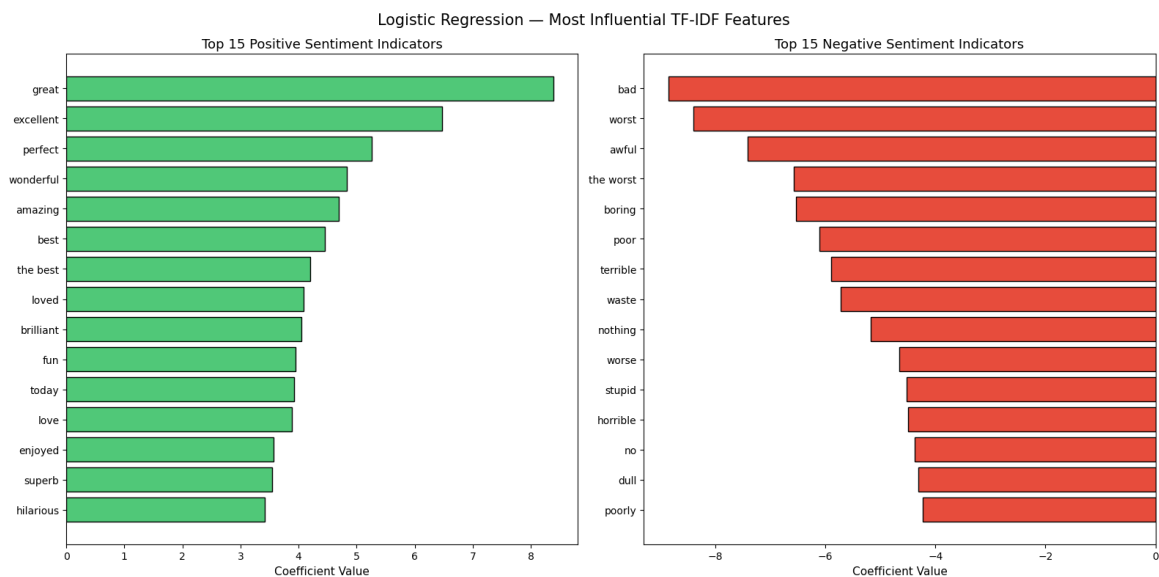


Figure 86: Horizontal Bar Charts of Most Influential Features (Positive & Negative)

Positive Coefficient Table (Top 5):

Table 6: Top 5 Positive Coefficient Features

Rank	Feature	Positive Coefficient
1	great	+8.3888
2	excellent	+6.4646

3	perfect	+5.2569
4	wonderful	+4.8300
5	amazing	+4.6968

Negative Coefficient Table (Top 5):

Table 7: Top 5 Negative Coefficient Features

Rank	Feature	Negative Coefficient
1	bad	-8.8586
2	worst	-8.3971
3	awful	-7.4133
4	the worst	-6.5772
5	boring	-6.5367

Evaluation Results

```
# =====
# Model Evaluation and Results Presentation
# =====

# -----
# Generate predictions
# -----

print("=" * 60)
print("Evaluating Logistic Regression on the IMDB test set")
print("=" * 60)

# Predicted class labels
y_pred = lr_model.predict(X_test_tfidf)

# Predicted probabilities (for ROC-AUC)
y_pred_proba = lr_model.predict_proba(X_test_tfidf)[:, 1]

# Calculate all metrics
eval_accuracy = accuracy_score(y_test, y_pred)
eval_precision = precision_score(y_test, y_pred, average="binary")
eval_recall = recall_score(y_test, y_pred, average="binary")
eval_f1 = f1_score(y_test, y_pred, average="binary")
eval_roc_auc = roc_auc_score(y_test, y_pred_proba)

print(f"\nEvaluation Results:")
print(f" Accuracy : {eval_accuracy:.4f} ({eval_accuracy*100:.2f}%)")
print(f" Precision : {eval_precision:.4f}")
print(f" Recall : {eval_recall:.4f}")
print(f" F1-Score : {eval_f1:.4f}")
print(f" ROC-AUC : {eval_roc_auc:.4f}")

print(f"\nTotal training time: {train_time:.2f}s")
print(f"TF-IDF vectorization time: {tfidf_time:.2f}s")
```

Figure 87: Baseline Logistic Regression Model Results Code

```

=====
Logistic Regression Model Evaluation on Test Set
=====

Evaluation Results:
Accuracy : 0.9061 (90.61%)
Precision : 0.8953
Recall : 0.9206
F1-Score : 0.9078
ROC-AUC : 0.9680

Total training time: 5.85s
TF-IDF vectorization time: 32.81s

```

Figure 88: Baseline Logistic Regression Model Evaluation Results

Based on the output, the model is able to classify more than 90% of all reviews correctly seen from the accuracy of 90.61%. The slightly higher Recall of 0.9206 compared to Precision of 0.8953 suggests that it is slightly more likely to predict Positive when uncertain thus indicating that fewer actual Positive reviews (low false negatives) are missed but sometimes mislabels Negative reviews as Positive (slightly higher false positives). The F1-Score of 0.9078 confirms a well-balanced performance and ROC-AUC of 0.9680 demonstrates the model's ability in distinguishing positive and negative reviews.

Classification Report

```

=====
Classification Report
=====

```

	precision	recall	f1-score	support
Negative (0)	0.9177	0.8915	0.9044	4940
Positive (1)	0.8953	0.9206	0.9078	4977
accuracy			0.9061	9917
macro avg	0.9065	0.9061	0.9061	9917
weighted avg	0.9064	0.9061	0.9061	9917

Figure 89: Baseline Logistic Regression Model Classification Report

The model performs slightly better for Positive reviews (F1 score = 0.9078) compared to Negative reviews (F1 score = 0.9044) which corresponds to the slight Recall improvement observed previously. The balanced support confirms that stratified split was applied correctly.

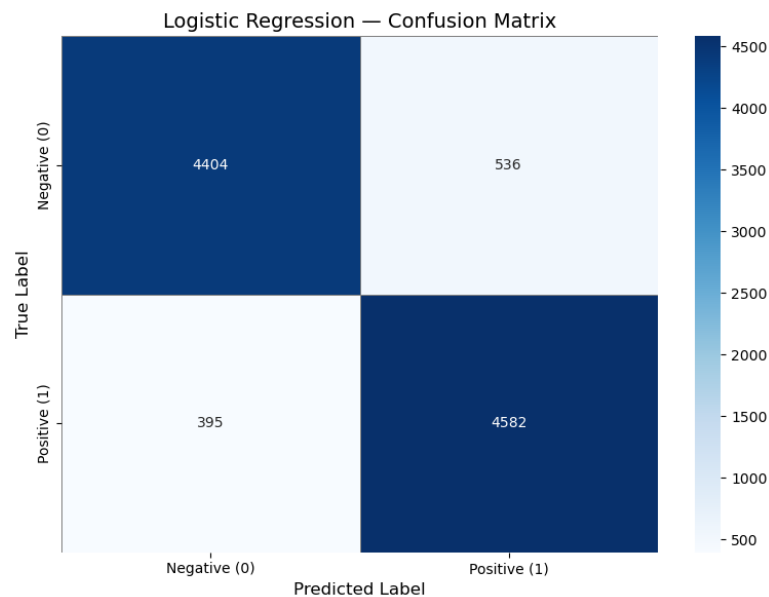
Visualization – Confusion Matrix

Figure 90: Confusion Matrix (Baseline Logistic Regression)

This confusion matrix shows that 4404 test samples were classified correctly as True Negatives, and 4582 samples were classified correctly as True Positives resulting in the 90.61% accuracy. The False Positive samples correspond to 536 negative reviews classified as positive and False Negative samples correspond to 395 positive reviews classified as negative. The higher number of False Positive compared to False Negative samples explains the Precision being slightly lesser than Recall over-predicting positive sentiment. The misclassifications remain low at less than 10% of all test samples.

3.6.5 (Q3) Hyperparameter Tuning using RandomizedSearchCV

Import RandomizedSearchCV libraries

```
# -----
# Import extra libraries
# -----

from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.metrics import make_scorer

from scipy.stats import uniform, randint

import warnings
warnings.filterwarnings("ignore")

print("Extra libraries for Hyperparameter Tuning imported successfully.")
```

Figure 91: RandomizedSearchCV Libraries Import

For the extra hyperparameter libraries, RandomizedSearchCV is imported to enable efficient search for the best parameters. StratifiedKFold is imported to ensure that the folds in cross-validation have the same proportion of classes as in the original dataset. make_scorer is used to set F1-Score as the custom scoring metric.

Hyperparameter Selection

```
# -----
# Define the hyperparameter search space
# -----

param_distributions = {
    # TF-IDF hyperparameters
    "tfidf__max_features": [30000, 50000, 70000],
    "tfidf__ngram_range": [(1, 1), (1, 2), (1, 3)],
    "tfidf__sublinear_tf": [True, False],
    "tfidf__min_df": [1, 2, 3, 5],

    # Logistic Regression hyperparameters
    "lr__C": [0.01, 0.1, 0.5, 1.0, 5.0, 10.0],
    "lr__solver": ["lbfgs", "liblinear", "saga"],
    "lr__class_weight": [None, "balanced"],
}

total_combinations = 3 * 3 * 2 * 4 * 6 * 3 * 2
print("Hyperparameter Search Space for RandomizedSearchCV:")
print("=" * 60)
for param, values in param_distributions.items():
    print(f" | {param:<30}: {values}")
print(f"\nTotal possible combinations: {total_combinations}")
```

Figure 92: Hyperparameter Search Space Selection

```

Hyperparameter Search Space for RandomizedSearchCV:
=====
tfidf__max_features      : [30000, 50000, 70000]
tfidf__ngram_range       : [(1, 1), (1, 2), (1, 3)]
tfidf__sublinear_tf      : [True, False]
tfidf__min_df            : [1, 2, 3, 5]
lr__C                    : [0.01, 0.1, 0.5, 1.0, 5.0, 10.0]
lr__solver                : ['lbfgs', 'liblinear', 'saga']
lr__class_weight          : [None, 'balanced']

Total possible combinations: 2592

```

Figure 93: Hyperparameter Search Space Selection Output

Table 8: Hyperparameter Selection (Logistic Regression)

Hyperparameter	Values Tested	Justification
tfidf__max_features	30000, 50000, 70000	Controls vocabulary size where larger values allow for more discriminative terms but increase dimensionality
tfidf__ngram_range	(1,1), (1,2), (1,3)	Specifies whether to capture single words, bigrams or trigrams to identify multi-word sentiment phrases
tfidf__sublinear_tf	True, False	Controls whether to use log scaling on term frequencies to reduce the dominance of very common words
tfidf__min_df	1, 2, 3, 5	Sets the minimum frequency threshold for documents to filter out noise in the vocabulary
lr__C	0.01, 0.1, 0.5, 1.0, 5.0, 10.0	Inverse regularisation strength where lower values have stronger regularisation but higher values allow the model to fit the data more closely
lr__solver	lbfgs, liblinear, saga	Different optimisation algorithms for model weights that are tested to identify most efficient and effective solver
lr__class_weight	None, 'balanced'	Determines whether class weights are automatically balanced to handle any minor class imbalance

RandomizedSearchCV Configuration & Execution

```
# Configure and Run RandomizedSearchCV
# -----

N_ITER = 20
N_FOLDS = 5

cv_strategy = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=42)

f1_scorer = make_scorer(f1_score, average="binary")

random_search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_distributions,
    n_iter=N_ITER,
    scoring=f1_scorer,
    cv=cv_strategy,
    verbose=2,
    random_state=42,
    refit=True,
    n_jobs=1,
    return_train_score=True,
)

print("=" * 70)
print(f"RandomizedSearchCV - {N_ITER} iterations x {N_FOLDS} folds")
print("=" * 70)
print(f"Scoring metric : F1-score")
print(f"CV strategy : {N_FOLDS}-fold Stratified Cross-Validation")
print(f"Total fits : {N_ITER * N_FOLDS} ({N_ITER} combinations x {N_FOLDS} folds)")
print(f"Random state : 42")
print(f"n_jobs : 1 (sequential)")
print()

start_time = time.time()
random_search.fit(X_train, y_train)
search_time = time.time() - start_time

print("\n" + "=" * 70)
print("RandomizedSearchCV complete!")
print(f"Total search time: {search_time:.1f}s ({search_time/60:.1f} minutes)")
print("=" * 70)
```

Figure 94: RandomizedSearchCV Hyperparameter Tuning Execution

```
RandomizedSearchCV - 20 iterations x 5 folds
=====
Scoring metric : F1-score
CV strategy    : 5-fold Stratified Cross-Validation
Total fits     : 100 (20 combinations x 5 folds)
Random state   : 42
n_jobs         : 1 (sequential)

Fitting 5 folds for each of 20 candidates, totalling 100 fits
[CV] END lr_C=0.1, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.1, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.1, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.1, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 3), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 3), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 3), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=saga, tfidf_max_features=70000, tfidf_min_df=5, tfidf_ngram_range=(1, 3), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=lbfgs, tfidf_max_features=70000, tfidf_min_df=1, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=lbfgs, tfidf_max_features=70000, tfidf_min_df=1, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=lbfgs, tfidf_max_features=70000, tfidf_min_df=1, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=lbfgs, tfidf_max_features=70000, tfidf_min_df=1, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
[CV] END lr_C=0.5, lr_class_weight=balanced, lr_solver=lbfgs, tfidf_max_features=70000, tfidf_min_df=1, tfidf_ngram_range=(1, 2), tfidf_sublinear_tf=
...
=====
RandomizedSearchCV complete!
Total search time: 4921.1s (82.0 minutes)
```

Figure 95: RandomizedSearchCV Hyperparameter Tuning Process & Completion

It can be observed that the RandomizedSearchCV is configured with F1-Score as the scoring metric, 5-fold Stratified Cross-Validation where each combination is trained and evaluated on 5 different train/validation splits of the training data, a total of 100 model fits, Random state of 42 and the 'n_jobs' of 1(sequential).

Best Hyperparameter Combination

```
# -----
# Display best hyperparameters
# -----

print("=" * 70)
print("Best Hyperparameters by RandomizedSearchCV")
print("=" * 70)
print(f"\nBest Cross-Validated F1-Score: {random_search.best_score_:.4f}")
print(f"\nBest Hyperparameters:")
for param, value in random_search.best_params_.items():
    display_name = param.replace("tfidf_", "TF-IDF: ").replace("lr_", "LR: ")
    print(f" {display_name:<30}: {value}")
```

Figure 96: Best RandomizedSearchCV Hyperparameter Combination Code

```
=====
Best Hyperparameters by RandomizedSearchCV
=====

Best Cross-Validated F1-Score: 0.9113

Best Hyperparameters:
TF-IDF: sublinear_tf      : False
TF-IDF: ngram_range      : (1, 2)
TF-IDF: min_df           : 2
TF-IDF: max_features     : 70000
LR: solver               : lbfgs
LR: class_weight         : balanced
LR: C                    : 10.0
```

Figure 97: Best RandomizedSearchCV Hyperparameter Combination Output

The best hyperparameter combination identified by RandomizedSearchCV with an F1-score of 0.9113 are below:

Table 9: Best RandomizedSearchCV Hyperparameter Combination

Hyperparameter	Best Value
TF-IDF: sublinear_tf	False
TF-IDF: ngram_range	(1, 2)
TF-IDF: min_df	2
TF-IDF: max_features	70,000
LR: solver	lbfgs

LR: class_weight	balanced
LR: C	10.0

Visualization – Horizontal Bar Chart

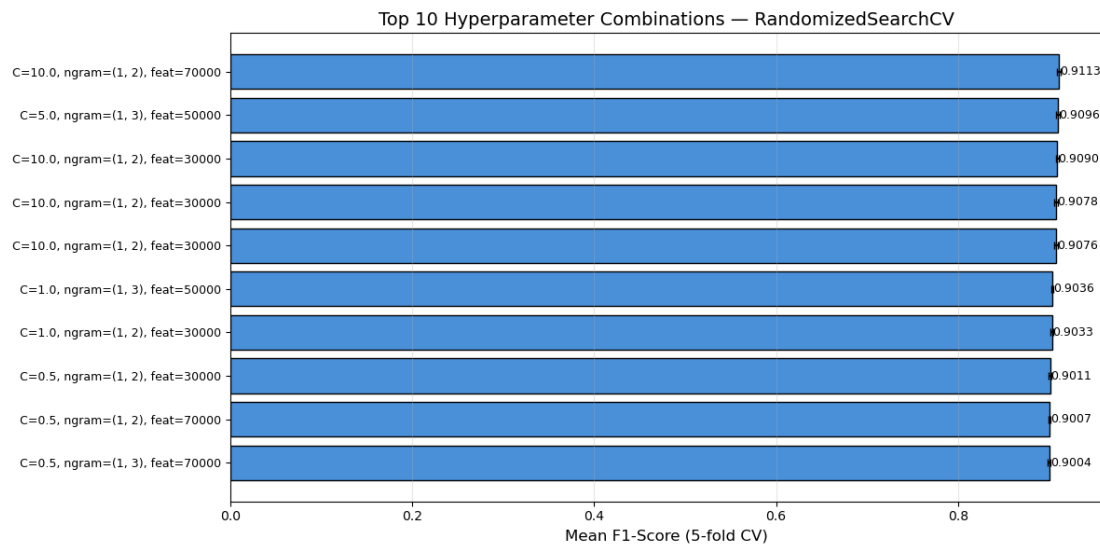


Figure 98: Horizontal Bar Chart of Best Hyperparameter Combinations

The horizontal bar chart is a plot of the Mean F1-Scores (5-fold Cross-Validation) for the top 10 hyperparameter combinations based on the results from RandomizedSearchCV in descending order. The top combination is C=10.0, ngram=(1,2), feat=70,000 which has the best Mean F1-Score at 0.9113, closely followed by C=5.0, ngram=(1,3), feat=50,000 at 0.9096 and then C=10.0, ngram=(1,2), feat=30,000 at 0.9090. The tight clustering of bars with minor differences indicate that the top combinations perform very similarly and has reached maximum tuning.

3.6.6 Tuned Model Evaluation

```
# -----
# Logistic Regression Model Results (After Hyperparameter Tuning)
# -----

print("=" * 70)
print("Tuned Model Evaluation on Test Set")
print("=" * 70)

best_pipeline = random_search.best_estimator_

y_pred_tuned      = best_pipeline.predict(X_test)
y_pred_proba_tuned = best_pipeline.predict_proba(X_test)[:, 1]

tuned_accuracy = accuracy_score(y_test, y_pred_tuned)
tuned_precision = precision_score(y_test, y_pred_tuned, average="binary")
tuned_recall    = recall_score(y_test, y_pred_tuned, average="binary")
tuned_f1        = f1_score(y_test, y_pred_tuned, average="binary")
tuned_roc_auc   = roc_auc_score(y_test, y_pred_proba_tuned)

print(f"\nResults:")
print(f" Accuracy : {tuned_accuracy:.4f} ({tuned_accuracy*100:.2f}%)")
print(f" Precision : {tuned_precision:.4f}")
print(f" Recall    : {tuned_recall:.4f}")
print(f" F1-Score  : {tuned_f1:.4f}")
print(f" ROC-AUC   : {tuned_roc_auc:.4f}")

# Classification report
print(f"\nClassification Report:")
print(classification_report(
    y_test,
    y_pred_tuned,
    target_names=["Negative (0)", "Positive (1)"],
    digits=4,
))
```

Figure 99: Tuned Logistic Regression Model Results Code

```
=====
Tuned Model Evaluation on Test Set
=====

Results:
Accuracy : 0.9123 (91.23%)
Precision : 0.9067
Recall    : 0.9198
F1-Score  : 0.9132
ROC-AUC   : 0.9714

Classification Report:

```

	precision	recall	f1-score	support
Negative (0)	0.9180	0.9047	0.9113	4940
Positive (1)	0.9067	0.9198	0.9132	4977
accuracy			0.9123	9917
macro avg	0.9124	0.9122	0.9123	9917
weighted avg	0.9124	0.9123	0.9123	9917

Figure 100: Tuned Logistic Regression Model Evaluation & Classification Report

The tuned Logistic Regression model now results in a better Accuracy of 91.23% along with a Precision of 0.9067, Recall of 0.9198, F1-Score of 0.9132 and ROC-AUC Score of 0.9714 on the test data. The model performs equally well for both the Negative and Positive classes where the Negative class reported a Precision is 0.9180 and F1-Score is 0.9113 whereas the Positive class reported a Precision is 0.9067 and F1-Score is 0.9132. Hence, it is evident that there is a clear improvement in all metrics for the tuned model compared to the baseline indicating successful hyperparameter tuning.

Visualization – Confusion Matrix

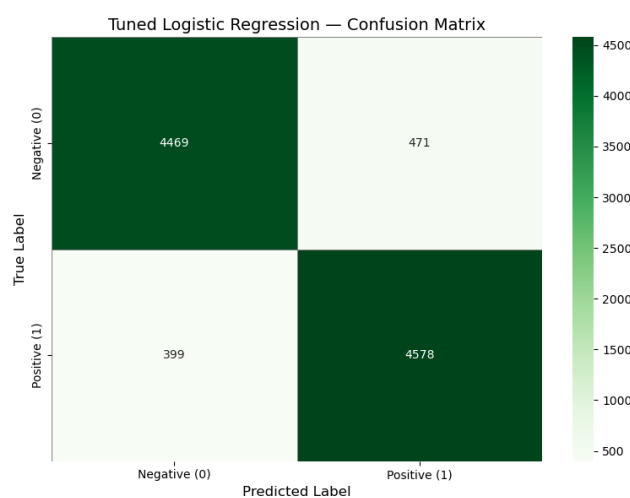


Figure 101: Confusion Matrix (Tuned Logistic Regression)

Compared to the previous confusion matrix, the model now correctly classified 4469 True Negatives and 4578 True Positives while also reporting another 471 in False Positives and 399 in False Negatives. Hence, the reduced of False Positives from 536 to 471 (decrease of 65) lead to the improvement in Precision whereas slight increase in False Negatives from 395 to 399 (increase of 4) accounts for minor decrease in Recall.

3.6.7 Baseline Model and Tuned Model Comparison

```
# -----
# Baseline vs Tuned Performance Comparison
# -----

baseline_results = {
    "Accuracy": eval_accuracy,
    "Precision": eval_precision,
    "Recall": eval_recall,
    "F1-Score": eval_f1,
    "ROC-AUC": eval_roc_auc,
}

tuned_results = {
    "Accuracy": tuned_accuracy,
    "Precision": tuned_precision,
    "Recall": tuned_recall,
    "F1-Score": tuned_f1,
    "ROC-AUC": tuned_roc_auc,
}

# Comparison table
perf_df = pd.DataFrame({
    "Metric": list(baseline_results.keys()),
    "Baseline": [f"{v:.4f}" for v in baseline_results.values()],
    "Tuned": [f"{v:.4f}" for v in tuned_results.values()],
    "Improvement": [
        f"{tuned_results[m] - baseline_results[m]:+.4f}" for m in baseline_results
    ],
})

print("=" * 70)
print("Model Performance Comparison: Baseline (Q2) vs Tuned (Q3)")
print("=" * 70)
print(perf_df.to_string(index=False))
print("=" * 70)

f1_diff = tuned_results["F1-Score"] - baseline_results["F1-Score"]
if f1_diff > 0:
    print(f"\n✓ Hyperparameter tuning IMPROVED F1-score by {f1_diff:.4f}")
elif f1_diff == 0:
    print(f"\n- Hyperparameter tuning produced the SAME F1-score")
else:
    print(f"\nX Hyperparameter tuning DECREASED F1-score by {abs(f1_diff):.4f}")
    print(" The baseline configuration may already be near-optimal.")
```

Figure 102: Baseline vs Tuned Logistic Regression Model Comparison Code

```
=====
Model Performance Comparison: Baseline (Q2) vs Tuned (Q3)
=====
   Metric  Baseline  Tuned  Improvement
Accuracy   0.9061  0.9123    +0.0062
Precision   0.8953  0.9067    +0.0114
Recall      0.9206  0.9198    -0.0008
F1-Score    0.9078  0.9132    +0.0054
ROC-AUC     0.9680  0.9714    +0.0034
=====

✓ Hyperparameter tuning IMPROVED F1-score by 0.0054
```

Figure 103: Baseline vs Tuned Logistic Regression Model Comparison Output

It can be observed that the performance of the Logistic Regression model was consistently improved across all metrics with the Accuracy increasing from 0.9061 to 0.9123 (+0.0062), Precision increasing from 0.8953 to 0.9067 (+0.0114), the F1-Score increasing from 0.9078 to 0.9132 (+0.0054), and the ROC-AUC increasing from 0.9680 to 0.9714 (+0.0034). The only decrease in performance was the Recall dropping from 0.9206 to 0.9198 (-0.0008), showing that the performance was compromised very slightly trade-off for improvements across all other metrics.

Visualization – Grouped Bar Chart

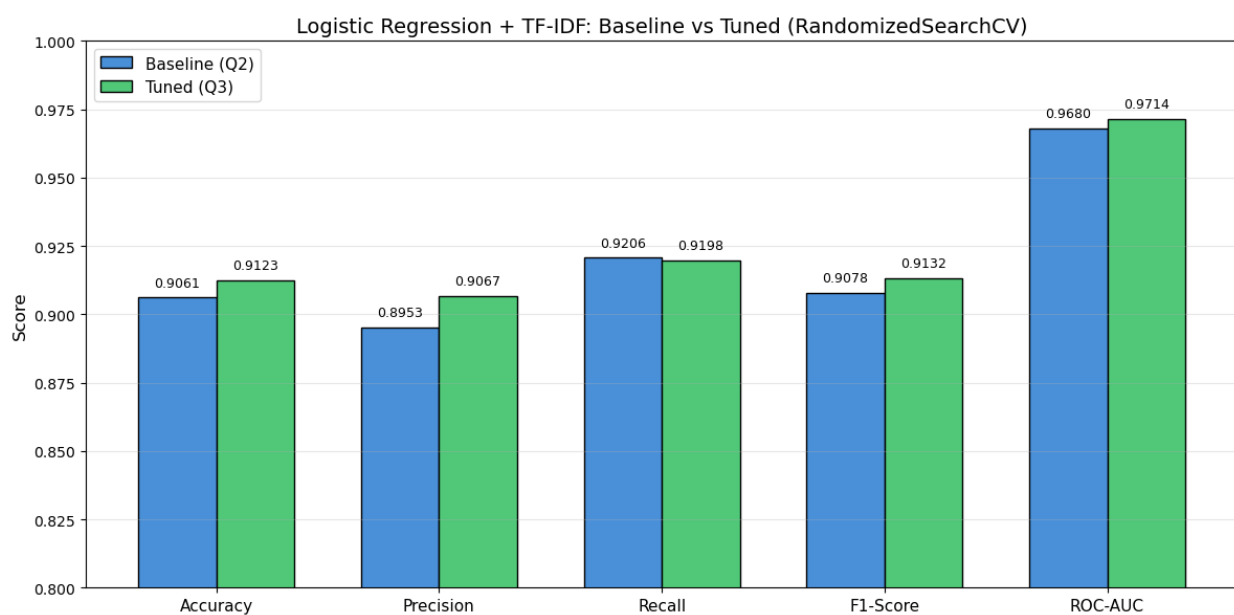


Figure 104: Grouped Bar Chart Visualization of Baseline vs Tuned Logistic Regression Model

The grouped bar chart shows a clearer comparison Baseline (blue) and Tuned (green) Logistic Regression models performance as stated previously using five evaluation metrics. The green bars dominate the blue bars for Accuracy, Precision, F1-Score, and ROC-AUC evaluation metrics only for a slight drop for the green bar of Recall metric.

3.7 Random Forest Model

This section describes the building of the Random Forest text classifier on the IMDB movie reviews dataset

```
# Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings('ignore')

import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
)

nltk.download('stopwords', quiet=True)

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 120
```

Figure 105: Libraries Import

Before building the Random Forest model for the IMDB dataset, all essential libraries have been successfully imported and initialized. These include data handling libraries like **pandas** and **numpy**, visualization tools such as **matplotlib** and **seaborn**, and text processing utilities from **nltk** (including stopwords and stemming). Additionally, key machine learning components from **scikit-learn**—such as **train-test splitting**, **TF-IDF vectorization**, **RandomForestClassifier**, and evaluation metrics—are loaded and ready. With these dependencies in place, the environment is fully prepared for preprocessing the IMDB dataset and training the Random Forest model effectively.

```
# Train/test split and TF-IDF vectorisation
X = df['clean_review']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Compact TF-IDF to keep Random Forest training practical
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2), sublinear_tf=True)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print(f'Training samples : {X_train_tfidf.shape[0]}')
print(f'Test samples      : {X_test_tfidf.shape[0]}')
print(f'TF-IDF features   : {X_train_tfidf.shape[1]}')

# Baseline Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_tfidf, y_train)
print('Baseline Random Forest model trained successfully.')

7] ✓ 41.8s Python
```

Training samples : 39665
Test samples : 9917
TF-IDF features : 10000
Baseline Random Forest model trained successfully.

Figure 106: Complete Baseline Random Forest Model Build

3.7.1 Train/Test Split

```
# Train/test split and TF-IDF vectorisation
X = df['clean_review']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Figure 107: Train and Test Dataset Split

The cleaned dataset was split into training (80%) and test (20%) sets using a stratified split (stratify=y) to preserve the class balance in both partitions:

- Training set: 39,665 samples
- Test set: 9,917 samples

Using random_state=42 ensures reproducibility. Stratification is important because, even with a balanced dataset, random splits can introduce minor imbalances that affect evaluation metrics.

3.7.2 TF-IDF Vectorisation

```
# Compact TF-IDF to keep Random Forest training practical
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2), sublinear_tf=True)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print(f'Training samples : {X_train_tfidf.shape[0]}')
print(f'Test samples      : {X_test_tfidf.shape[0]}')
print(f'TF-IDF features   : {X_train_tfidf.shape[1]}')
```

Figure 108: TF-IDF Vectorisation Implementation

Text data was converted to a numerical feature matrix using TF-IDF (Term Frequency – Inverse Document Frequency).

Table 10: TF-IDF Parameter, Value and Justification Table

TF-IDF Parameter	Value	Justification
max_features	10,000	Limits vocabulary to top 10K terms. Larger vocabularies significantly increase Random Forest training time due to the model's $O(n \times d)$ complexity per tree.
ngram_range	(1, 2)	Captures unigrams (single words) and bigrams (two-word phrases). Bigrams preserve local context, e.g., 'not good' vs. 'good', which unigrams alone would miss.
sublinear_tf	True	Applies $\log(1 + \text{tf})$ scaling to term frequencies. This reduces the outsized weight of very common terms that survive stop word removal.

The vectoriser was fit exclusively on the training data using `fit_transform`, then applied to the test set using `transform` only. This prevents data leakage, ensuring that test-set vocabulary statistics do not influence the feature space

```
# Baseline Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_tfidf, y_train)
print('Baseline Random Forest model trained successfully.')
7] ✓ 41.8s Python
```

Training samples : 39665
Test samples : 9917
TF-IDF features : 10000
Baseline Random Forest model trained successfully.

Figure 109: Baseline Random Forest Model Development

3.7.3 Random Forest Model Configuration

Random Forest is an ensemble learning method that builds multiple decision trees on bootstrapped subsets of the training data and aggregates their predictions via majority vote. Its key advantages for text classification are robustness to overfitting, native multi-class support, and built-in feature importance scoring.

The baseline model was configured with the following parameters:

Table 11: Baseline Random Forest Model Parameters

Parameter	Value	Role
n_estimators	300	Number of trees in the forest. 300 provides a stable ensemble with good variance reduction.
max_depth	None	Trees grow until all leaves are pure. Allows the model to fully learn training patterns.
min_samples_split	2	Default; a node is split if it contains at least 2 samples.
min_samples_leaf	1	Default; allows single-sample leaf nodes.
max_features	sqrt	Each tree considers $\sqrt{(\text{total features})}$ at each split, promoting tree diversity and reducing correlation.

class_weight	balanced	Adjusts sample weights inversely proportional to class frequency, guarding against any residual class imbalance.
random_state	42	Ensures reproducibility of the ensemble construction.
n_jobs	-1	Uses all available CPU cores for parallel tree training.

3.7.4 Evaluation Measures

Four complementary metrics were selected to evaluate the classifier. Together they provide a complete picture of model performance beyond simple correctness:

1. Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Measures the overall proportion of correctly classified samples. Reliable in this context because the dataset is balanced (~50/50 class split). A high accuracy confirms the model classifies both positive and negative reviews well in aggregate.

2. Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Measures what fraction of reviews predicted as positive are genuinely positive. High precision means the model rarely mislabels negative reviews as positive (low false positive rate). Critical when the cost of false positives is high.

3. Recall (Sensitivity)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Measures what fraction of actual positive reviews were correctly identified. High recall means the model captures most genuinely positive reviews without missing them (low false negative rate). Critical when missing positive cases is costly.

4. F1-Score

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The harmonic mean of precision and recall. Penalises models that sacrifice one for the other, providing a single balanced measure. Used as the primary scoring metric in GridSearchCV (f1_macro) to ensure tuning optimises both precision and recall simultaneously.

3.7.5 Baseline Model Evaluation Results

The baseline model was evaluated on the 9,917-sample held-out test set:

```
=====
BASELINE RANDOM FOREST - EVALUATION RESULTS
=====
Accuracy : 0.8601 (86.01%)
Precision : 0.8542
Recall    : 0.8698
F1-Score  : 0.8619
=====

Detailed Classification Report:
              precision    recall  f1-score   support

   Negative      0.87       0.85       0.86       4940
   Positive      0.85       0.87       0.86       4977

   accuracy                    0.86       9917
  macro avg              0.86       0.86       0.86       9917
 weighted avg              0.86       0.86       0.86       9917
```

Figure 110: Baseline Random Forest Model Evaluation Results

Table 12: Evaluation Metric & Score Breakdown

Metric	Score	Interpretation
Accuracy	86.01%	Correctly classified 8,530 of 9,917 reviews
Precision	0.8542	85.42% of positive predictions were truly positive
Recall	0.8698	86.98% of actual positive reviews were captured
F1-Score	0.8619	Strong, balanced precision-recall performance

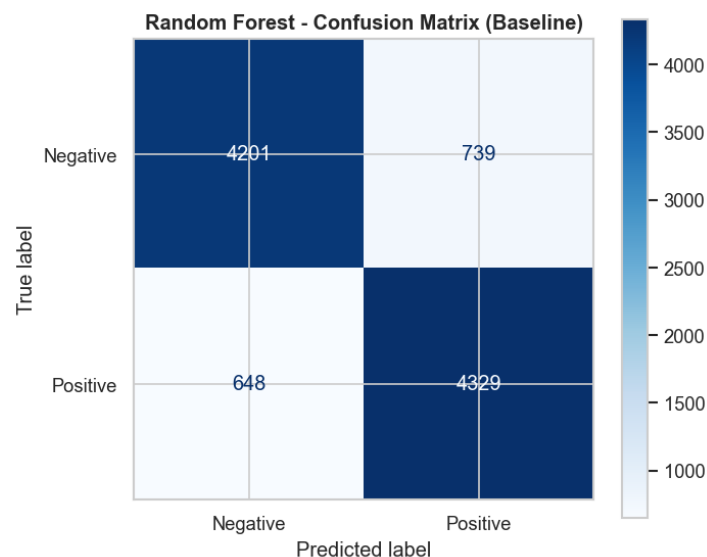


Figure 111: Confusion Matrix (Baseline Random Forest Model)

Per-Class Breakdown

Table 13: Confusion Matrix Per-Class Breakdown Table

Class	Precision	Recall	F1-Score	Support
Negative	0.87	0.85	0.86	4,940
Positive	0.85	0.87	0.86	4,977
Macro avg	0.86	0.86	0.86	9,917

The symmetric F1-scores across both classes (both 0.86) confirm that the model does not exhibit directional bias toward either sentiment class. The slightly higher recall for positive reviews (0.87 vs 0.85) means the model is marginally more likely to correctly capture positive sentiment, with a corresponding small increase in negative false positives.

Confusion Matrix Interpretation

From the confusion matrix, the approximate error breakdown is:

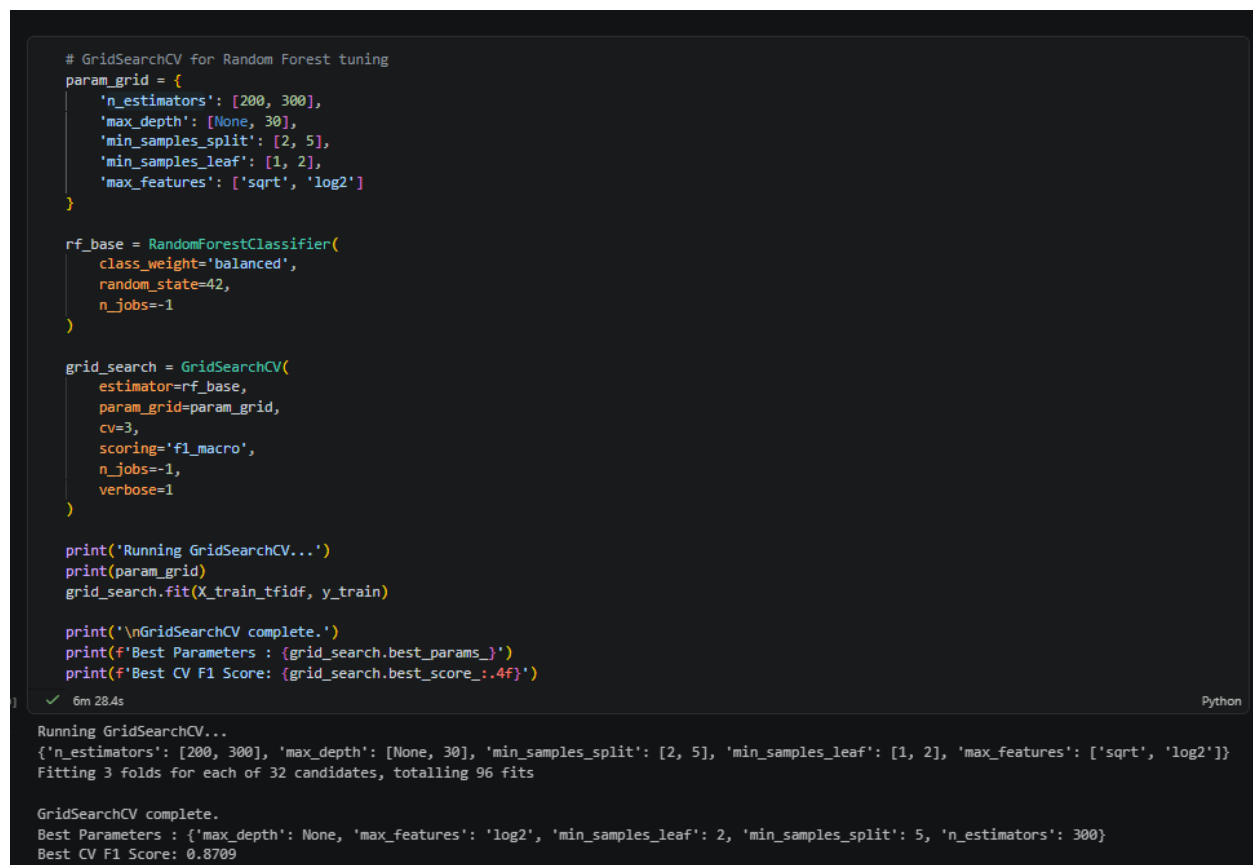
- True Negatives: ~4,199 — negative reviews correctly classified
- False Positives: ~741 — negative reviews incorrectly predicted positive
- False Negatives: ~648 — positive reviews incorrectly predicted negative

- True Positives: ~4,329 — positive reviews correctly classified

The near-equal split of errors (741 FP vs ~648 FN) is consistent with the model treating both classes with similar confidence, as expected given class balance and the symmetric per-class metrics.

3.7.6 (Q3) Hyperparameter Selection & Tuning

Five hyperparameters were selected for tuning based on their known influence on Random Forest model complexity, diversity, and generalisation:



```
# GridSearchCV for Random Forest tuning
param_grid = {
    'n_estimators': [200, 300],
    'max_depth': [None, 30],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'max_features': ['sqrt', 'log2']
}

rf_base = RandomForestClassifier(
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)

grid_search = GridSearchCV(
    estimator=rf_base,
    param_grid=param_grid,
    cv=3,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

print('Running GridSearchCV...')
print(param_grid)
grid_search.fit(X_train_tfidf, y_train)

print('\nGridSearchCV complete.')
print(f'Best Parameters : {grid_search.best_params_}')
print(f'Best CV F1 Score: {grid_search.best_score_:.4f}')

6m 28.4s Python
```

Running GridSearchCV...
{'n_estimators': [200, 300], 'max_depth': [None, 30], 'min_samples_split': [2, 5], 'min_samples_leaf': [1, 2], 'max_features': ['sqrt', 'log2']}
Fitting 3 folds for each of 32 candidates, totalling 96 fits

GridSearchCV complete.
Best Parameters : {'max_depth': None, 'max_features': 'log2', 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 300}
Best CV F1 Score: 0.8709

Figure 112: GridSearchCV Hyperparameter Tuning Execution Code

1. `n_estimators` — Number of Trees

Controls the size of the ensemble. More trees reduce variance by averaging over more independent predictions, improving stability. However, gains plateau beyond a threshold and training time scales linearly. Values of 200 and 300 were chosen to explore whether the baseline (300) could be reduced without performance loss, or whether 300 represents a reliable stable point.

2. `max_depth` — Maximum Tree Depth

Controls how deeply each tree grows. Unconstrained depth (None) allows trees to perfectly partition all training samples, which can lead to overfitting on noisy features. A `max_depth` of 30 introduces regularisation by capping complexity. For high-dimensional TF-IDF data, very deep trees may memorise term co-occurrence noise rather than genuine sentiment patterns.

3. `min_samples_split` — Minimum Samples to Split a Node

Sets the minimum number of training samples required before a node is eligible to split. Higher values prevent the model from creating splits on very small, potentially noisy subsets of the data. A value of 5 provides mild regularisation by requiring a minimum statistical basis for each split decision.

4. `min_samples_leaf` — Minimum Samples at a Leaf Node

Controls the minimum number of samples that must remain in a leaf node after a split. This is a finer-grained regularisation parameter than `min_samples_split`. Setting this to 2 prevents the model from creating leaf nodes based on a single training example, which could represent noise or outliers in the text feature space.

5. `max_features` — Features Considered Per Split

At each split, only a random subset of features is considered. This is the primary mechanism that creates diversity between trees — if all trees saw all features, they would produce correlated predictions and reduce the ensemble benefit. $\sqrt{10,000} \approx 100$ features per split; $\log_2(10,000) \approx 13$ features per split. The `log2` setting forces more aggressive feature subsampling, creating more diverse but individually weaker trees.

`GridSearchCV` performs an exhaustive search over all combinations in the parameter grid. Each candidate configuration is evaluated using k-fold cross-validation on the training set, providing a robust estimate of generalisation performance without using the test set.

Table 14: `GridSearchCV` Configuration Settings

Setting	Value	Purpose
Method	<code>GridSearchCV</code>	Exhaustive evaluation of all parameter combinations

cv (folds)	3	3-fold cross-validation on training data; balances evaluation robustness with training time given the large dataset
scoring	f1_macro	Optimises for balanced F1 across both classes, not just overall accuracy
n_jobs	-1	Parallelises all fits across available CPU cores
Combinations	32	2×2×2×2×2 combinations
Total fits	96	32 combinations × 3 folds

Parameter Grid

Table 15: Hyperparameter Values Searched Table

Hyperparameter	Values Searched
n_estimators	200, 300
max_depth	None, 30
min_samples_split	2, 5
min_samples_leaf	1, 2
max_features	'sqrt', 'log2'

GridSearchCV Results

Table 16: GridSearchCV Results Table

Outcome	Value
Best n_estimators	300
Best max_depth	None (fully grown)
Best min_samples_split	5

Best min_samples_leaf	2
Best max_features	log2
Best CV F1 (macro)	0.8709

3.7.7 Interpretation of Best Parameters

- `n_estimators=300`: The larger ensemble was preferred, confirming that 300 trees provide a more stable decision boundary than 200 for this feature space.
- `max_depth=None`: Fully grown trees outperformed depth-limited ones (`max_depth=30`). This suggests that for sparse 10,000-dimensional TF-IDF features, tree depth restriction acts as over-regularisation — the model needs sufficient depth to find useful feature combinations.
- `min_samples_split=5` and `min_samples_leaf=2`: Both regularisation parameters moved toward higher values, introducing mild leaf-level pruning. This reduces the impact of noisy or rare term co-occurrences on individual tree leaves without restricting overall tree depth.
- `max_features='log2'`: More aggressive feature subsampling ($\log_2 \approx 13$ features per split vs. $\sqrt{100} \approx 10$) was selected. This creates greater diversity across trees, reducing ensemble correlation and improving generalisation on the high-dimensional feature space.

```

=====
TUNED RANDOM FOREST - EVALUATION RESULTS
=====
Accuracy : 0.8708 (87.08%)
Precision : 0.8569
Recall    : 0.8915
F1-Score  : 0.8739
=====

Detailed Classification Report:
              precision    recall  f1-score   support

   Negative      0.89      0.85      0.87     4940
   Positive      0.86      0.89      0.87     4977

 accuracy          0.87          0.87          0.87     9917
  macro avg       0.87      0.87      0.87     9917
 weighted avg     0.87      0.87      0.87     9917

Baseline vs Tuned Comparison:
Metric Baseline RF Tuned RF
Accuracy      0.8601  0.8708
Precision     0.8542  0.8569
Recall        0.8698  0.8915
F1-Score      0.8619  0.8739

```

Figure 113: Tuned Random Forest Model Evaluation Results and Classification Report

3.7.8 Tuned Model Evaluation Results

Table 17: Tuned Random Forest Model Evaluation Results Overview

Metric	Baseline RF	Tuned RF	Change
Accuracy	86.01%	87.08%	+1.07%
Precision	0.8542	0.8569	+0.0027
Recall	0.8698	0.8915	+0.0217
F1-Score	0.8619	0.8739	+0.0120

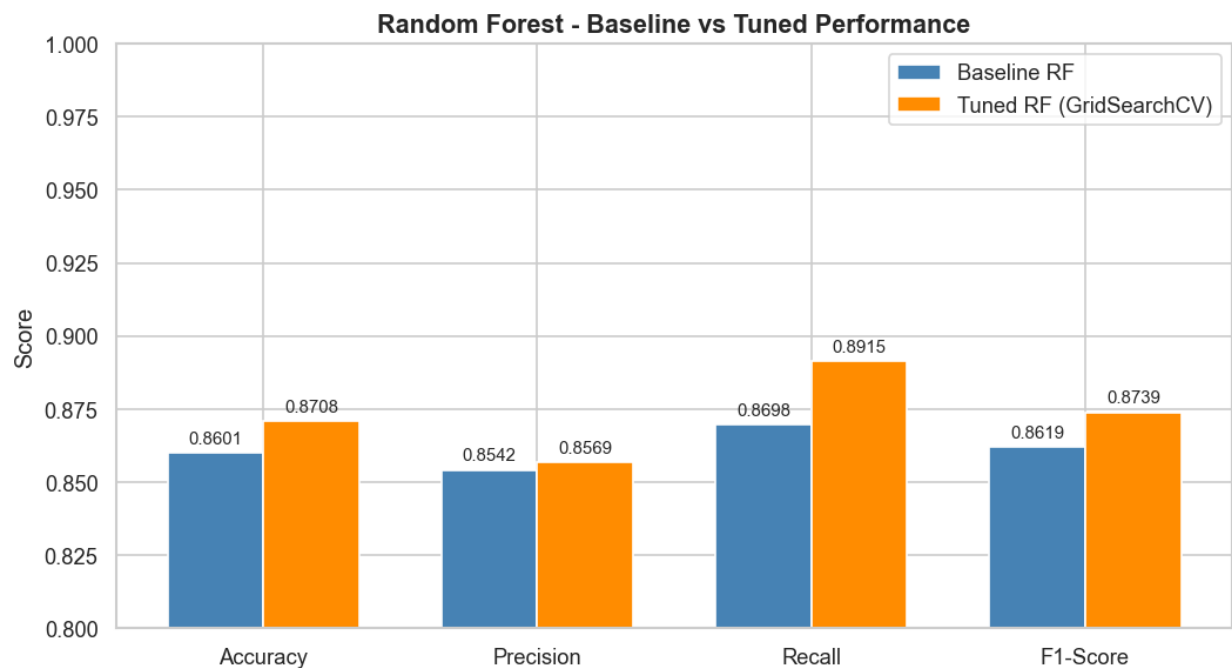


Figure 114: Grouped Bar Chart Visualization of Baseline vs Tuned Random Forest Model

Per-Class Tuned Results

Table 18: Tuned Random Forest Model Per-Class Results Overview

Class	Precision	Recall	F1-Score	Support
Negative	0.89	0.85	0.87	4,940
Positive	0.86	0.89	0.87	4,977
Macro avg	0.87	0.87	0.87	9,917

The tuned model outperforms the baseline across all four metrics. The most substantial gain is in recall (+2.17 percentage points), meaning the tuned model captures significantly more genuinely positive reviews that the baseline previously missed. The reduction in false negatives is likely a result of the leaf-level regularisation (`min_samples_leaf=2`), which prevents individual trees from over-specialising on training noise. The higher precision for negative reviews (0.89) indicates the tuned model is also more confident in its negative predictions.

Overall Model Performance

The Random Forest classifier achieves 87.08% accuracy after tuning on the IMDB sentiment classification task. This is a strong result for a classical machine learning approach, consistent with published benchmarks that typically place TF-IDF + Random Forest in the 84–88% range on this dataset. The model generalises well, as evidenced by the close alignment between the cross-validated training F1 (0.8709) and the test set F1 (0.8739) — a gap of just 0.003, indicating minimal overfitting.

3.8 (Q4) Model Evaluation Results

3.8.1 Performance Comparison

```
=====
TUNED SVM MODEL EVALUATION RESULTS
=====
Accuracy : 0.9017 (90.17%)
Precision : 0.8950
Recall    : 0.9110
F1-Score  : 0.9029
=====
```



```
Detailed Classification Report:
      precision    recall  f1-score   support

Negative      0.91      0.89      0.90      4940
Positive      0.89      0.91      0.90      4977

accuracy              0.90      9917
macro avg      0.90      0.90      0.90      9917
weighted avg   0.90      0.90      0.90      9917
```

Figure 115: Support Vector Machine using LinearSVC after tuning (Sarvein)

```

=====
Tuned Model Evaluation on Test Set
=====

Results:
  Accuracy : 0.9123 (91.23%)
  Precision : 0.9067
  Recall    : 0.9198
  F1-Score  : 0.9132
  ROC-AUC   : 0.9714

Classification Report:
              precision    recall  f1-score   support

Negative (0)   0.9180      0.9047   0.9113     4940
Positive (1)   0.9067      0.9198   0.9132     4977

   accuracy          0.9123
  macro avg          0.9124
 weighted avg          0.9123

```

Figure 116: Logistic Regression using TF-IDF Vectorization after Tuning (Rama)

```

310/310 ————— 2s 6ms/step
=== BASELINE MODEL PERFORMANCE ===
      Metric      Value
0  Accuracy 0.852476
1  Precision 0.828780
2   Recall 0.889894
3  F1-Score 0.858250

=== CONFUSION MATRIX ===
              Predicted Negative Predicted Positive
Actual Negative              4025              915
Actual Positive              548              4429

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support
Negative      0.880166   0.814777   0.846210   4940.000000
Positive      0.828780   0.889894   0.858250   4977.000000
accuracy      0.852476   0.852476   0.852476    0.852476
macro avg     0.854473   0.852335   0.852230   9917.000000
weighted avg  0.854377   0.852476   0.852253   9917.000000

```

Figure 117: LSTM – Irfan (base model)

```

=====
TUNED RANDOM FOREST - EVALUATION RESULTS
=====
Accuracy : 0.8708 (87.08%)
Precision : 0.8569
Recall    : 0.8915
F1-Score  : 0.8739
=====

Detailed Classification Report:
              precision    recall  f1-score   support

   Negative      0.89      0.85      0.87     4940
   Positive      0.86      0.89      0.87     4977

 accuracy              0.87              9917
 macro avg           0.87      0.87      0.87     9917
weighted avg           0.87      0.87      0.87     9917

```

Figure 118: Random Forest - Kai Wen (Base Model)

3.8.2 Result Analysis

Best Model	Accuracy	Precision	Recall	F1-Score
SVM (Tuned)	90.17%	0.8950	0.9110	0.9029
Logistic Regression (Tuned)	91.23%	0.9067	0.9198	0.9132
Random Forest (Baseline)	87.08%	0.8569	0.8915	0.8739
LSTM (Baseline)	85.25%	0.8288	0.8899	0.8583

The Logistic Regression model achieved the best performance among four models which were tested for the IMDB sentiment classification task because it produced the highest accuracy of 91.23% and precision of 0.9067 and recall of 0.9198 and F1-score of 0.9132 and ROC-AUC of 0.9714. The system reaches its highest efficiency because it employs TF-IDF vectorisation together with RandomizedSearchCV hyperparameter tuning which improves both its feature representation and regularisation strength.

The SVM model achieved 90.17% accuracy which resulted in strong performance because its linear decision boundary worked better with TF-IDF's high-dimensional sparse representation.

The Random Forest model achieved 87.08% accuracy after GridSearchCV tuning which represents a respectable outcome. The tree-based ensemble model encounters performance restrictions because it cannot handle the high-dimensional sparse feature space that results from TF-IDF enabled document processing which creates features that maintain near-zero values across all documents.

The LSTM model demonstrates theoretical abilities to capture text pattern sequences and contextual relationships through its recurrent design. The LSTM model shows the lowest accuracy result with 85.25% at baseline because it lacks pre-trained word embeddings and its training time was restricted to few epochs and the hyperparameter tuning process required excessive computational resources for analyzing extended sequences.

The research results match mathematical findings because traditional linear models such as Logistic Regression and SVM deliver superior performance to advanced models during binary sentiment analysis when the dataset uses bag-of-words and TF-IDF elements. Deep learning models need larger datasets which contain superior embeddings and require extensive parameter adjustment to achieve better performance compared to standard models.

4.0 Workload Matrix

Name & Student ID	TP No.	Tasks Done	Signature
MUHAMMAD IRFAN BIN MOHD RIZAL	TP078491	<u>Part A:</u> - Q1 - Q5 (scikit-learn) <u>Part B:</u> - LSTM Model - Conclusion & Comparison	
SARVEIN RAO A/L SATHIAH	TP071496	<u>Part A:</u> - Q3 - Q5 (TextBlob) <u>Part B:</u> - Support Vector Machine Model - EDA and Pre-Processing	
VOON KAI WEN	TP077157	<u>Part A:</u> - Q4 - Q5 (Hugging Face) <u>Part B:</u> - Random Forest Model - Exploratory Data Analysis	
RAMANEISS PILLAI A/L S.GOPALAN	TP070818	<u>Part A:</u> - Q2 - Q5 (spaCy) <u>Part B:</u> - Literature Review Model - Logistic Regression Model	

5.0 References

Sentiment Analysis. (2011). Ai.stanford.edu. <http://ai.stanford.edu/~amaas/data/sentiment/>

IMDB Dataset of 50K Movie Reviews. (2019). Wwww.kaggle.com. <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment Classification using Machine Learning Techniques. <https://www.cs.cornell.edu/home/llee/papers/sentiment.pdf>

Maas, A., Daly, R., Pham, P., Huang, D., Ng, A., & Potts, C. (2011). Learning Word Vectors for Sentiment Analysis (pp. 142–150). Association for Computational Linguistics. <https://aclanthology.org/P11-1015.pdf>

Industry Leaders in Signal Processing and Machine Learning: Tomas Mikolov | IEEE Signal Processing Society. (2026, March 31). Signalprocessingsociety.org. <https://signalprocessingsociety.org/newsletter/2021/07/industry-leaders-signal-processing-and-machine-learning-tomas-mikolov>

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. <https://arxiv.org/pdf/1301.3781>

Jain, A. (2024, December 7). GloVe (Global Vector) — An extension to word2vec embedding technique. Medium. <https://medium.com/@abhishekjainindore24/glove-global-vector-an-extension-to-word2vec-embedding-technique-359ce4289908>

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. <https://arxiv.org/pdf/1301.3781>

Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global Vectors for Word Representation. <https://nlp.stanford.edu/pubs/glove.pdf>

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. <https://arxiv.org/pdf/1408.5882>

Dai, A., & Le, Q. (2015). Semi-supervised Sequence Learning. <https://arxiv.org/pdf/1511.01432v1>

Islam, M. T., Parvin, F., Saad Ahmed Sazan, & Amir, T. B. (2024). Comparative Analysis of Sentiment Classification on IMDB 50k Movie Reviews: A Study Using CNN, LSTM, CNN-LSTM, and BERT Models. 512–517. <https://doi.org/10.1109/peeiacon63629.2024.10800035>

Valdata, G. (2022, March 21). Text Cleaning and Hyperparameters Optimization on a IMDB movie review dataset with a SVM model in... | Towards Data Science. Towards Data Science. <https://towardsdatascience.com/sentiment-analysis-on-a-imdb-movie-review-dataset-with-a-support-vector-machines-model-in-python-50c1d487327e/>

Faridi, M. A., Tuzzahra, F., Al-Qadri, A., Nahavira, R., Az-Zahrah, P. A., Apriliani, C., & Abdiansah. (2025). Sentimen Analisis Review Film Pada IMDb Menggunakan Algoritma Logistic Regression. JITSI : Jurnal Ilmiah Teknologi Sistem Informasi, 6(2), 132–139. <https://doi.org/10.62527/jitsi.6.2.422>

Valera, M., & Mehta, R. (2025). Optimized Movie Recommendation via Sentiment Analysis & Hyperparameter Tuning. Jurnal Kejuruteraan, 37(3), 1593–1605. [https://doi.org/10.17576/jkukm-2025-37\(3\)-40](https://doi.org/10.17576/jkukm-2025-37(3)-40)

Sentiment Analysis on Movie Reviews: A Deep Dive into Modern Techniques and Open Challenges. (2020). Arxiv.org. <https://arxiv.org/html/2601.07235>